TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS · CAPÍTULO 3

Banco de Dados e SQL

SGBD, PostgreSQL, pgAdmin e os primeiros comandos

```sql
SELECT nome_aluno
FROM alunos
WHERE cidade_aluno = 'Recife';
```

PROFESSOR
Tarik Ponciano

CAPÍTULO 3 • BANCO DE DADOS

Roteiro

01 **Banco de dados e SGBD**
O que são, por que existem e o que fazem por você

02 **PostgreSQL**
O SGBD da disciplina: no seu computador ou na nuvem (Aiven)

03 **pgAdmin**
Conectar, criar o banco e as tabelas, inserir dados pela interface

04 **SQL: consulta e manipulação**
SELECT, INSERT, UPDATE e DELETE no banco da escola

★ **Consolidação**
Do briefing ao banco: a Barbearia Navalha de Ouro

NESTE MATERIAL

Exemplo explicado
Um caso resolvido passo a passo.

Prática
Individual e digital, no seu PostgreSQL, com o professor acompanhando.

Código SQL
Testado no PostgreSQL, com o resultado que aparece na tela.

ONDE ESTAMOS

Do desenho ao banco de verdade

CAPÍTULO 1 · CONCEITUAL

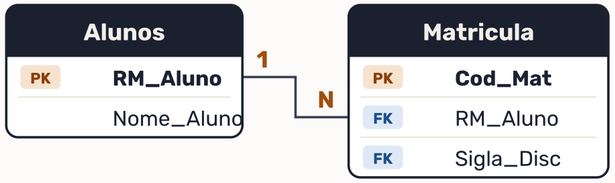
DER



O que existe e como se liga.

CAPÍTULO 2 · LÓGICO

MER na 3FN



Tabelas, atributos, PK e FK — sem repetição.

CAPÍTULO 3 · FÍSICO · HOJE

PostgreSQL + SQL

```
CREATE TABLE alunos (
  rm_aluno   integer PRIMARY KEY,
  nome_aluno varchar(100) NOT NULL
);
```

O modelo vira um banco de verdade, com tipos de dados — e recebe dados.

Hoje o MER da escola sai do papel: vamos criá-lo num **SGBD**, colocar dados nele e conversar com ele em **SQL**.

SEÇÃO 01

Banco de dados e SGBD

01

O que são, por que existem e o que um SGBD faz pelos seus dados.

Banco de dados?



Imagem: meme popular da internet • autor desconhecido.

Antes que alguém pergunte: não, não é este.

Este é um **banco** feito de **dados** — de jogo. Confortável? Nem um pouco.

O nosso guarda outro tipo de dado: nomes, notas, pedidos, pagamentos, músicas favoritas… e é sobre ele que vamos falar.

Aquecimento: quantos bancos de dados — dos de verdade — você já usou hoje, desde que acordou?

01 • BANCO DE DADOS E SGBD

O que é um banco de dados

> Um banco de dados é uma **coleção de dados inter-relacionados**, que representa informações sobre um **assunto específico**.
>
> Definição adaptada de Korth e Silberschatz.

1. **Dados relacionados.** Um assunto só, com dados que se ligam — não uma pilha aleatória de informações.
2. **Com um propósito.** Projetado para atender um grupo de usuários e de aplicações.
3. **Espelho do mundo real.** Uma mudança na realidade — uma nova matrícula — vira uma mudança no banco.

VOCÊ USA TODOS OS DIAS

Contatos do celular
nome, telefone, foto

Loja virtual
produtos, clientes, pedidos

Streaming de música
músicas, playlists, histórico

Sistema da escola
alunos, disciplinas, notas

01 • BANCO DE DADOS E SGBD

Por que bancos de dados importam

Quase todo sistema que você vai programar guarda dados num banco: o app do banco, o PIX, o e-commerce, o prontuário do posto de saúde, o sistema acadêmico. Por quê?

Guardar sem perder
Os dados continuam lá depois que o programa fecha — e depois de anos.

Muita gente ao mesmo tempo
Milhares de pessoas comprando, pagando ou se matriculando no mesmo segundo.

Achar rápido
Encontrar um cliente entre milhões em milissegundos.

Dados confiáveis
Nota 15, data 31/02 ou matrícula de aluno que não existe? Recusado.

Seguro
Cada usuário vê e altera só o que pode.

Fonte única
O site, o app e o sistema interno leem os mesmos dados.

Uma planilha guarda dados. Um **sistema de banco de dados** guarda, protege, organiza e entrega esses dados para muitos usuários e aplicações ao mesmo tempo.

01 • BANCO DE DADOS E SGBD

O vocabulário do banco relacional

alunos			
rm_aluno	nome_aluno	data_nasc	cidade_aluno
101	André Silva	2008-03-14	Recife
102	Carla Motta	2007-11-02	Olinda
103	Bruno Lima	2009-06-25	**Recife**

TABELA — O assunto guardado: **alunos**. Vem da entidade do MER.

COLUNA • CAMPO — Uma característica: **nome_aluno**. Vem do atributo.

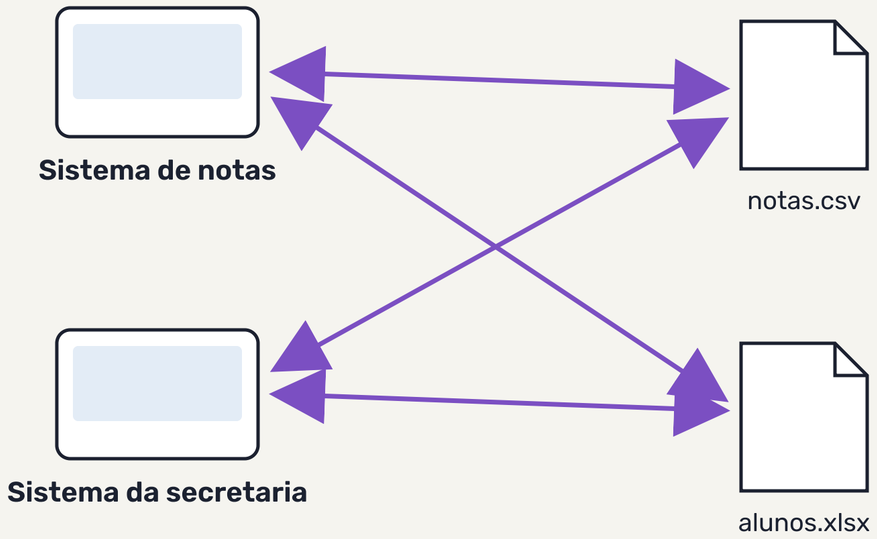
LINHA • REGISTRO — Um aluno específico: a **Carla**.

VALOR — O dado de uma célula: **Recife**.

Banco relacional: os dados ficam em tabelas que se ligam por chaves — exatamente o MER que você já sabe fazer.

Esquema é a estrutura (tabelas e colunas). **Dados** são as linhas que entram nela.

Antes do SGBD: cada programa com seus arquivos



cada programa lê e grava direto nos arquivos

Mudou a estrutura?

Uma coluna nova no arquivo obriga a alterar **todos** os programas — até os que nem usam essa coluna.

Dados repetidos e divergentes

Cada sistema guarda a sua cópia do aluno. Qual delas está certa?

Dois ao mesmo tempo

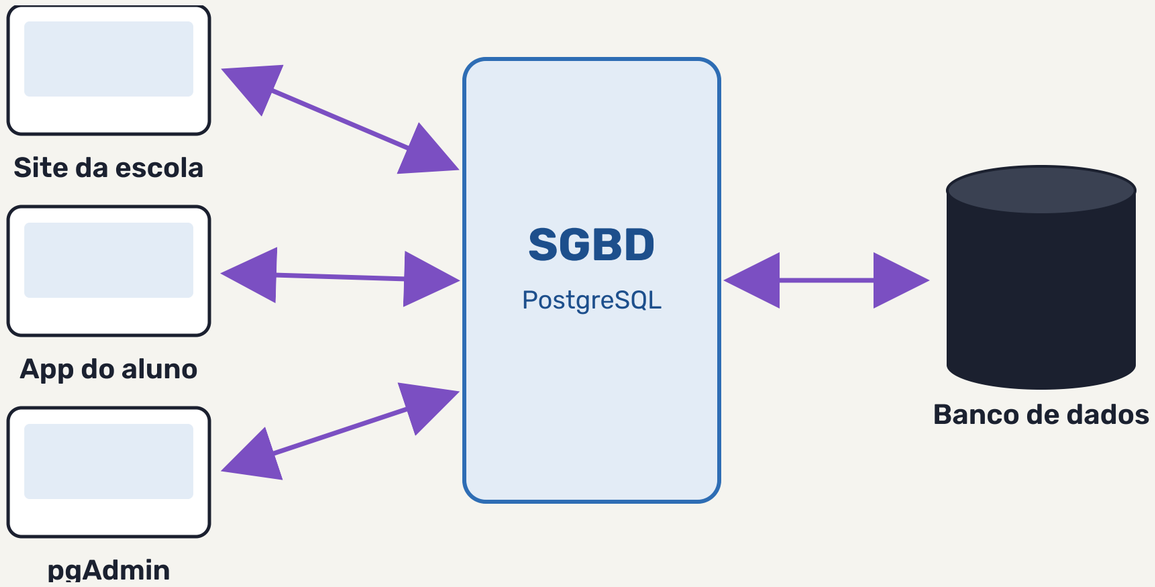
A secretaria e o professor salvam juntos: o último apaga o trabalho do outro.

Sem controle

Quem abre o arquivo vê e altera tudo — notas, CPFs, endereços.

O que é um SGBD

SGBD — Sistema Gerenciador de Banco de Dados — é o software que fica **entre os dados e as aplicações**: guarda, protege e entrega os dados.



- As aplicações **nunca mexem nos arquivos**: pedem os dados ao SGBD.
- Cada programa **vê só o que precisa**, sem saber como os dados estão gravados.
- Mudou a estrutura? Só muda quem usa aquela parte: **independência dos dados**.
- Exemplos: **PostgreSQL**, MySQL, SQL Server, Oracle, SQLite.

01 • BANCO DE DADOS E SGBD

O que o SGBD faz por você

Integridade

Regras que o dado precisa cumprir: tipo, PK, FK, campo obrigatório, faixa de valores.

`ex.: data 31/02 recusada`

Concorrência

Muitos usuários lendo e gravando ao mesmo tempo, sem um estragar o trabalho do outro.

`ex.: duas compras do último item`

Segurança

Usuários, senhas e permissões: quem pode ver, inserir, alterar ou excluir.

`ex.: estagiário só consulta`

Backup e recuperação

Cópias e registro das operações para voltar depois de uma falha.

`ex.: o servidor desligou`

Desempenho

Índices e otimização para achar um registro entre milhões rapidamente.

`ex.: busca por CPF`

Linguagem padrão

Todos os SGBDs relacionais conversam em SQL.

`ex.: SELECT … FROM …`

Consistência: os dados obedecem às regras. **Validade**: cada valor pertence ao conjunto de valores possíveis — um telefone não pode ser “Residencial”.

EXEMPLO EXPLICADO • INTEGRIDADE

A planilha aceita tudo; o SGBD não

NA PLANILHA • TUDO ENTRA

rm	nome	data_nasc	telefone	nota
101	André Silva	14/03/2008	98888-1111	8,5
101	Outro André	02/05/2009	97777-2222	7
103	Bruno Lima	31/02/2009	Residencial	oito

A planilha aceitou um RM repetido, uma data que não existe, um telefone que é texto e uma nota por extenso. Ninguém avisou.

NO POSTGRESQL • O SGBD RECUSA

RM repetido — a PK não aceita repetição

```
ERROR: duplicate key value violates unique constraint "alunos_pkey"
```

Data 31/02 — o tipo date só aceita datas reais

```
ERROR: date/time field value out of range: "2008-02-31"
```

Nota “oito” — a coluna nota é numérica

```
ERROR: invalid input syntax for type numeric: "oito"
```

Essas regras são **criadas por você** ao montar as tabelas — o tipo de cada coluna, a PK, os campos obrigatórios. A seção 03 mostra como.

PRÁTICA 1 • 4 MIN • INDIVIDUAL

Banco, SGBD ou aplicação?

Classifique cada item como **B** (banco de dados — os dados), **S** (SGBD — o software que gerencia) ou **A** (aplicação — usa os dados).

a PostgreSQL

b Os alunos, notas e faltas da escola

c O app em que o professor lança as notas

d MySQL

e pgAdmin

f O catálogo de produtos de uma loja virtual

g O site da loja virtual

h SQLite

FORMATO DA RESPOSTA

```
a) S
```

Dica: pergunte “isso **são** dados, **guarda** dados ou **usa** dados?”

PENSE E RESPONDA O pgAdmin é o banco de dados? Se você desinstalar o pgAdmin, os dados somem?

PRÁTICA 2 • 5 MIN • INDIVIDUAL

Qual recurso do SGBD resolve?

Para cada situação, diga **qual recurso do SGBD** resolve o problema — e como.

1. Dois caixas vendem, no mesmo segundo, a última unidade de um produto.
2. O estagiário pode consultar os alunos, mas não pode ver os salários.
3. O disco do servidor queimou durante a noite.
4. Alguém digitou a nota 15 numa prova que vale 10.
5. O sistema tentou matricular o aluno 999, que não existe.
6. A busca por CPF ficou lenta com 2 milhões de clientes.

RECURSOS

- INTEGRIDADE
- CONCORRÊNCIA
- SEGURANÇA
- BACKUP E RECUPERAÇÃO
- DESEMPENHO

```
1) Concorrência — o SGBD …
```

PENSE E RESPONDA Qual dessas situações uma planilha compartilhada também resolveria?

SEÇÃO 02

PostgreSQL

O SGBD da disciplina: como ele funciona e como colocar um servidor no ar — no seu computador ou na nuvem.

02

02 • POSTGRESQL

PostgreSQL: o SGBD da disciplina

O QUE É

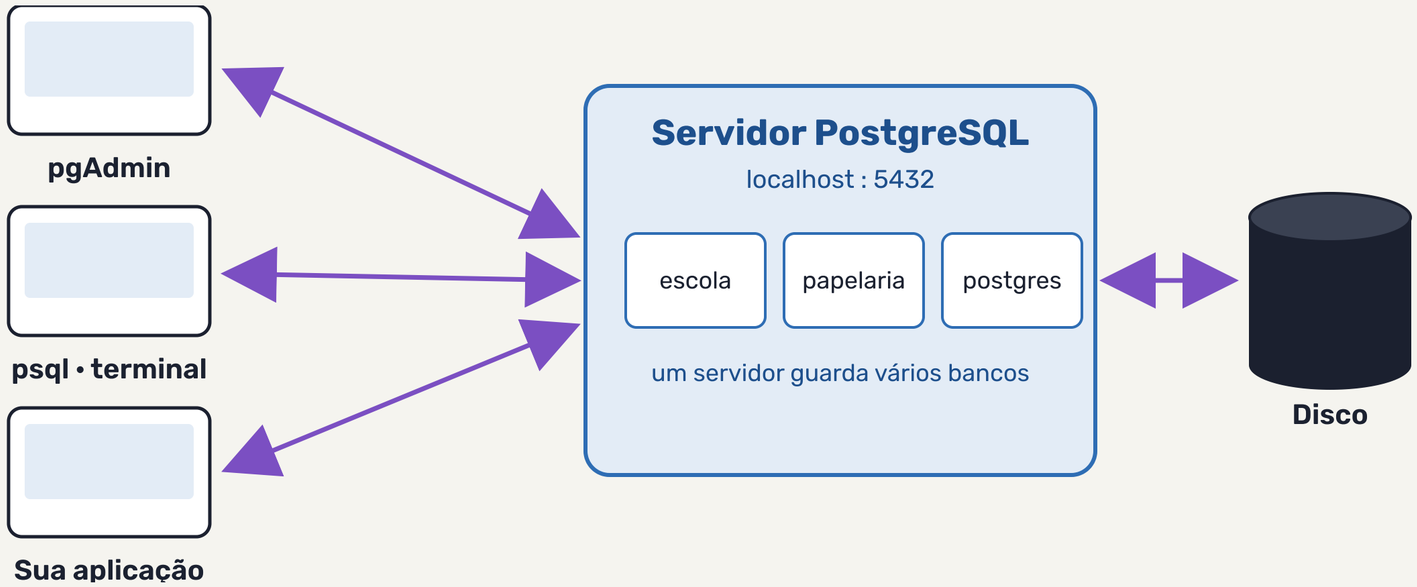
- Um **SGBD relacional**: dados em tabelas ligadas por chaves.
- **Código aberto e gratuito**, inclusive para uso comercial.
- Roda em **Windows, Linux e macOS**.
- Segue o **padrão SQL**: o que você aprender aqui vale em outros SGBDs.
- Nasceu na Universidade da Califórnia, em Berkeley, nos anos 1980 — e é mantido por uma comunidade global até hoje.

POR QUE ELE NESTA DISCIPLINA

- Você instala **em casa, sem pagar nada**.
- Está disponível como serviço nas grandes nuvens — AWS, Google Cloud, Azure — e na **Aiven**, que vamos usar.
- É um dos SGBDs **mais usados pelo mercado**: aparece em vagas de back-end e de dados.
- Tem tipos de dados ricos: datas, valores monetários, textos, JSON.
- Vem com o **pgAdmin**, uma interface gráfica para começar sem decorar comandos.

No dia a dia, todo mundo chama de **“Postgres”**. Na documentação e nas vagas: **PostgreSQL**.

Cliente e servidor



Servidor

Um processo que fica rodando e é o **único** que mexe nos arquivos do banco.

Clientes

pgAdmin, psql e a sua aplicação **pedem** os dados ao servidor — mesmo no mesmo computador.

Para conectar

host, **porta**, **banco**, **usuário** e **senha**. Guarde essas cinco informações.

02 • POSTGRESQL

Local ou na nuvem?

O servidor pode estar em dois lugares. O pgAdmin e o SQL são os mesmos — **só muda o endereço da conexão**.

	LOCAL • INSTALADO	NUVEM • AIVEN
Onde o servidor roda	No seu computador	Num datacenter da Aiven
O que instalar	PostgreSQL (o instalador já traz o pgAdmin)	Só o pgAdmin, para acessar
Precisa de internet?	Não	Sim
Acessar de outro lugar	Só naquela máquina	De casa, do laboratório, do celular
Custo	Gratuito	Plano Free, sem cartão de crédito
Quando escolher	Computador pessoal, com permissão de administrador	Laboratório sem permissão para instalar; trabalhar de vários lugares

Nesta disciplina, as duas valem. Muita gente usa as duas: local em casa, Aiven no laboratório — mas aí são dois bancos diferentes, cada um com seus dados.

02 • POSTGRESQL

Instalação local no Windows

1 Baixe o instalador

postgresql.org/download → **Windows** → *Download the installer* (EDB). Escolha a versão mais recente.

2 Componentes

Deixe marcados **PostgreSQL Server**, **pgAdmin 4** e **Command Line Tools**. O Stack Builder é opcional.

3 Pastas

Aceite a pasta de instalação e a pasta de dados (*data*) sugeridas.

4 Senha do superusuário

Crie a senha do usuário `postgres` — o administrador do servidor. **Anote.**

5 Porta

Mantenha `5432`, a porta padrão do PostgreSQL.

6 Concluir

Locale: deixe o padrão. Avance até o fim e desmarque “Launch Stack Builder”.

⚠ **A senha do `postgres`** é a chave do servidor. Esquecê-la é o problema mais comum da aula.

Linux: pacote `postgresql` da distribuição. **macOS:** instalador da EDB ou Postgres.app.

02 • POSTGRESQL

Conferindo a instalação

DEU CERTO SE…

1. **O serviço está rodando.** Em **Serviços** do Windows, procure `postgresql-x64-NN` (NN é a versão) com status “Em execução”.
2. **O pgAdmin abre.** No menu Iniciar, abra o **pgAdmin 4**. Em *Servers* já aparece **PostgreSQL NN**.
3. **A senha funciona.** Clique no servidor: ele pede a senha do `postgres`. Conectou? Pronto.

PROBLEMAS COMUNS

Porta 5432 ocupada

Já existe outra instalação. O instalador sugere 5433 — use essa porta ao conectar.

Sem permissão de administrador

Comum no laboratório. Use a Aiven: nada para instalar além do pgAdmin.

Esqueceu a senha

Peça ajuda ao professor antes de reinstalar. E, da próxima vez, anote num lugar seguro.

02 • POSTGRESQL

Banco na nuvem com a Aiven

A **Aiven** oferece PostgreSQL gerenciado na nuvem, com um plano gratuito: o servidor fica no ar e você só conecta.

1 Crie a conta

Em **console.aiven.io**, com e-mail, Google ou GitHub. O plano gratuito não pede cartão.

2 Novo serviço

No projeto: **Services** → **Create service** → **PostgreSQL**.

3 Plano Free

Escolha o tier **Free**. A nuvem e a região são definidas pela Aiven.

4 Dê um nome

Ex.: `pg-escola`. Clique em **Create service**.

5 Espere ficar pronto

O status muda de *Rebuilding* para **Running** em alguns minutos.

6 Copie a conexão

Na página **Overview** do serviço: host, porta, usuário, senha, banco e SSL (ou **Quick connect**).

Para acessar: instale só o pgAdmin, em `pgadmin.org/download`.

Serviço gratuito **parado por muito tempo** pode ser desligado. Religue pelo console.

02 • POSTGRESQL

Os dados de conexão

A Aiven entrega a conexão num endereço só, a **Service URI**. Cada pedaço vira um campo no pgAdmin:

```
postgres://avnadmin:SENHA@pg-escola-turma.a.aivencloud.com:12691/defaultdb?sslmode=require
```

CAMPO NO PGADMIN	AIVEN (EXEMPLO)	LOCAL
Host name/address	`pg-escola-turma.a.aivencloud.com`	`localhost`
Port	`12691` (a do seu serviço)	`5432`
Maintenance database	`defaultdb`	`postgres`
Username	`avnadmin`	`postgres`
Password	a do console da Aiven	a que você criou na instalação
SSL mode	`require`	padrão (não precisa mexer)

⚠ **A senha é sua.** Não mande print da conexão no Discord nem suba a URI para o GitHub: quem tem a URI entra no seu banco.

PRÁTICA 3 • 15 MIN • INDIVIDUAL

Seu servidor no ar

Escolha um caminho e deixe o seu servidor PostgreSQL pronto para a próxima seção.

CAMINHO A • LOCAL

1. Instale o PostgreSQL com o pgAdmin.
2. Confira o serviço e a senha do `postgres`.

CAMINHO B • AIVEN

1. Crie a conta e o serviço PostgreSQL Free.
2. Espere o status **Running** e instale o pgAdmin.

PRONTO QUANDO

- O servidor está rodando (serviço do Windows ou **Running** na Aiven).
- Você anotou **host, porta, banco, usuário e senha** num lugar seguro.
- O pgAdmin está instalado e abre.

PENSE E RESPONDA Quem escolheu a Aiven consegue usar o mesmo banco em casa? E quem instalou no computador do laboratório?

SEÇÃO 03

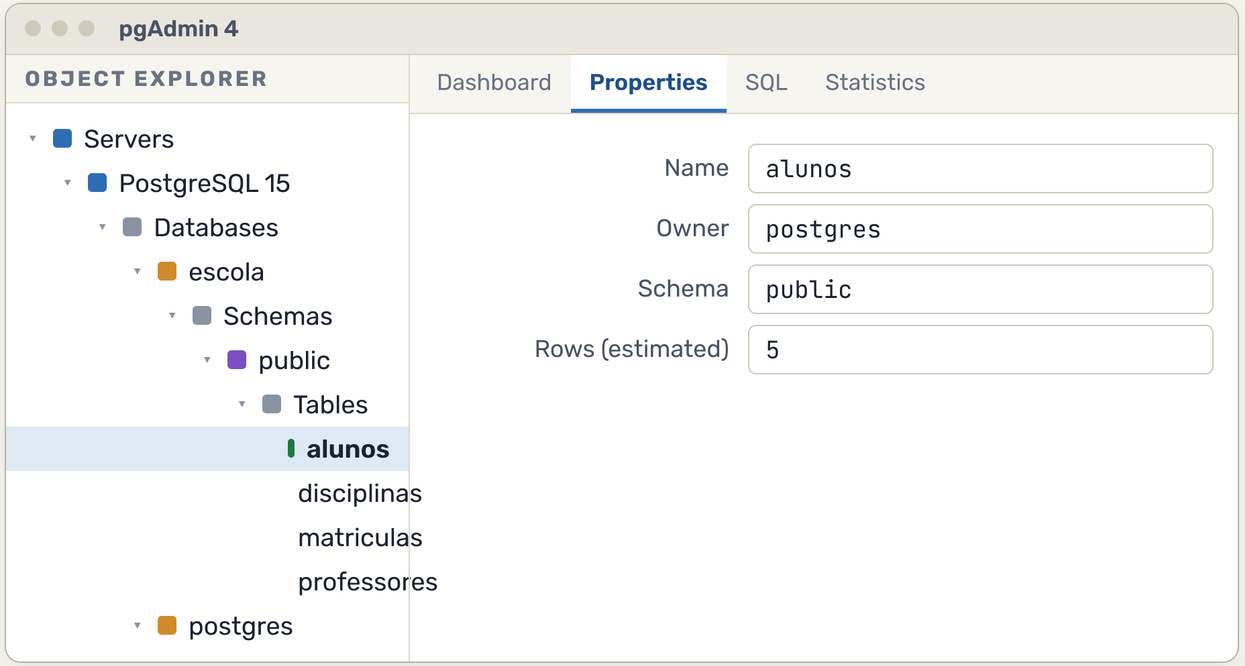
pgAdmin

Conectar ao servidor, transformar o MER em tabelas de verdade e colocar dados nelas — primeiro juntos, depois sozinho.

03

03 • PGADMIN

O pgAdmin



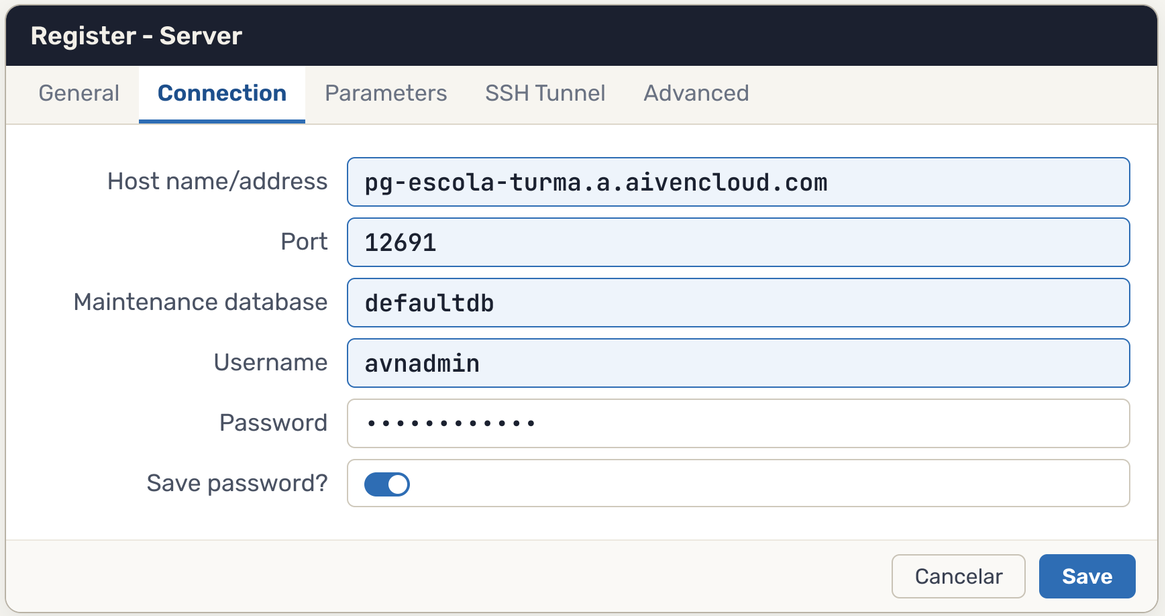
Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

- A **interface gráfica oficial** do PostgreSQL: um **cliente**, como vimos.
- Conecta a servidores locais e na nuvem.
- Cria bancos e tabelas, mostra e edita dados e roda SQL na **Query Tool**.
- Já vem no instalador do PostgreSQL. Para a Aiven, instale só ele.

⚠ O pgAdmin **não é o banco**: é a janela para ele. Na 1ª vez, ele pede uma **senha mestra**, que protege as senhas salvas.

03 • PGADMIN

Conectar ao servidor



Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

1. **Abrir o cadastro.** Clique com o botão direito em **Servers** → **Register** → **Server…**
2. **General. Name**: um apelido, ex.: `Aiven - escola`.
3. **Connection.** Host, porta, banco, usuário e senha — os dados que você anotou.
4. **Parameters.** Na Aiven: **SSL mode** = `require`. (Em versões antigas, aba **SSL**.)
5. **Save.** O servidor aparece na árvore. Clique nele para conectar.

Instalação local? O servidor **PostgreSQL NN** já vem cadastrado: é só clicar e digitar a senha do `postgres`.

03 • PGADMIN

Quem guarda quem

- **Servidor** — `PostgreSQL 15 ou Aiven` — o SGBD rodando, local ou na nuvem
  - **Banco de dados** — `escola` — um servidor guarda vários bancos
    - **Schema** — `public` — uma “pasta” dentro do banco — usamos a public
      - **Tabela** — `alunos` — as tabelas do seu MER
        - **Colunas e restrições** — `rm_aluno, nome_aluno… · PK` — a estrutura de cada tabela

No pgAdmin: **Servers** → **servidor** → **Databases** → **escola** → **Schemas** → **public** → **Tables**.

Cada ferramenta abre **num banco**. Abriu a Query Tool no banco `postgres`? A tabela `alunos` “não existe” lá.

Tipos de dados essenciais

TIPO	GUARDA	NA ESCOLA
`integer`	número inteiro	`rm_aluno, faltas, carga_horaria`
`numeric(p, s)`	decimal exato: p dígitos, s depois da vírgula	`nota numeric(4,2) → 8.50`
`varchar(n)`	texto de até n caracteres	`nome_aluno varchar(100)`
`char(n)`	texto de tamanho fixo	`cpf_aluno char(11)`
`text`	texto sem limite definido	`observacoes`
`date`	data (ano-mês-dia)	`data_nasc → '2008-03-14'`
`timestamp`	data e hora	`'2026-02-02 19:30'`
`boolean`	verdadeiro ou falso	`ativo → true`
`serial`	inteiro gerado automaticamente (1, 2, 3…)	`id de uma tabela nova`

ESTRATÉGIA: COMO ESCOLHER

- Vai fazer **conta**? Número: `integer` ou `numeric`.
- Tem **zero à esquerda** ou não faz conta (CPF, CEP, telefone)? **Texto**.
- É data? `date` — **nunca** texto.
- Dinheiro e nota: `numeric`, que não arredonda errado.

Lista completa: postgresql.org/docs/current/datatype.html

03 • PGADMIN

Restrições: as regras do banco

Restrições são as **regras** que o SGBD confere a cada inserção e alteração. Nesta primeira implementação, vamos usar quatro — todas fáceis de achar no pgAdmin:

RESTRIÇÃO	O QUE GARANTE	NA ESCOLA	ONDE NO PGADMIN
PRIMARY KEY	identifica cada linha: única e nunca vazia	`rm_aluno`	aba **Columns** → *Primary key?*
NOT NULL	campo obrigatório	`nome_aluno`	aba **Columns** → *Not NULL?*
UNIQUE	não repete — mas não é a PK	`cpf_aluno`	aba **Constraints** → *Unique*
DEFAULT	valor automático quando nada é informado	`faltas → 0`	lápis da coluna → *Constraints* → *Default value*

E o **tipo** de cada coluna já é uma regra: um campo `date` não aceita “31/02”, um `integer` não aceita “oitenta”.

FOREIGN KEY e **CHECK** também são restrições — ficam para um próximo passo.

03 • PGADMIN

PK própria ou PK composta?

PK COMPOSTA • SABER QUE EXISTE

Duas ou mais colunas que, **juntas**, identificam a linha. Sozinhas elas repetem — o RM 101 aparece duas vezes, BD também —, mas o **par** nunca se repete.

PK = (RM_ALUNO, SIGLA_DISC)

rm_aluno	sigla_disc	nota
101	BD	8.50
101	LOG	7.00
102	BD	9.50

```
PRIMARY KEY (rm_aluno, sigla_disc)
```

```
Repetir (101, BD) → ERROR: duplicate key … already exists.
```

No pgAdmin: *Primary key?* ligado nas duas colunas. Aparece em livros e sistemas prontos.

PK PRÓPRIA • O PADRÃO DA TURMA

Toda tabela ganha **um código só dela**, numa coluna só: `cod_mat`, `cod_emprestimo`, `cod_item`.

PK = COD_MAT

cod_mat	rm_aluno	sigla_disc	nota
1	101	BD	8.50
2	101	LOG	7.00
3	102	BD	9.50

- **Uma coluna no WHERE:** `WHERE cod_mat = 6`.
- **Mesmo padrão** em todas as tabelas.
- O aluno trocou de disciplina? A **PK não muda**.

⚠ **Nesta disciplina: sempre PK própria.** Só atenção a um detalhe — com ela, o banco não impede o mesmo aluno duas vezes na mesma disciplina. Confira antes de lançar.

03 • PGADMIN • ATENÇÃO

Antes de criar: maiúsculas e aspas

```
-- criada no pgAdmin com o nome "Clientes"
SELECT * FROM Clientes;
```

```
ERROR: relation "clientes" does not exist
```

```
SELECT * FROM "Clientes";   -- só funciona com aspas
```

- O PostgreSQL **converte para minúsculas** todo nome escrito sem aspas.
- Se a tabela foi criada com maiúscula, você vai precisar de **aspas duplas** em todo comando.
- **Regra da turma:** nomes em minúsculas, com `_`, sem acento e sem espaço: `alunos`, `nome_aluno`.

⚠ **Outros tropeços comuns:** esquecer de salvar a grade (F6) • Query Tool aberta no banco errado • vírgula em vez de ponto no decimal • data em formato brasileiro.

CONSTRUÇÃO GUIADA • ESCOLA

Vamos construir o banco da escola — juntos

O ROTEIRO

1. Do MER ao projeto físico
2. O projeto físico da escola
3. Criar o banco escola
4. Tabela professores: colunas, PK e UNIQUE
5. Tabelas disciplinas e alunos
6. Tabela matriculas: código próprio e DEFAULT
7. Conferir a aba SQL
8. Lançar os dados pela grade
9. O banco cobra as regras?

COMO FUNCIONA

O professor mostra cada passo no projetor e **todos fazem junto**, no próprio servidor — local ou Aiven.

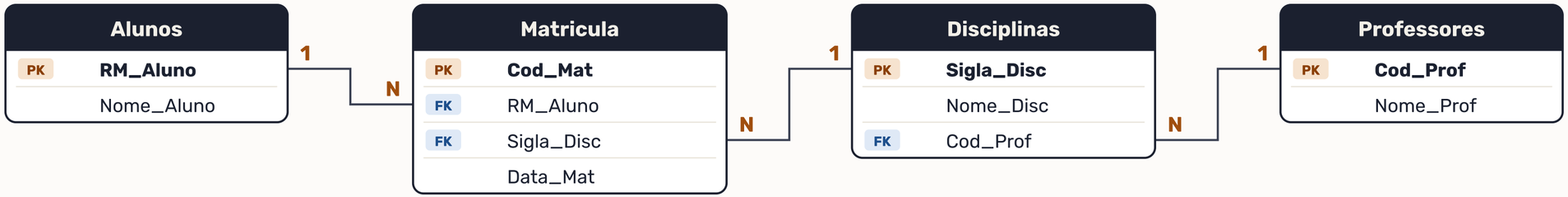
No fim, cada um terá o banco `escola` pronto, com dados — é nele que vamos aprender SQL.

Depois, na **Prática 4**, você repete o processo sozinho, numa situação nova: a **biblioteca** da escola.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 1 DE 9

Do MER ao projeto físico

O MER DA ESCOLA • CAPÍTULO 2



1. **Nomes em minúsculas.** `Alunos.RM_Aluno` vira `alunos.rm_aluno`: sem maiúsculas, acentos ou espaços.

2. **Um tipo para cada coluna.** RM é número inteiro; nome é texto; data é `date`; nota é decimal.

3. **As regras simples.** Uma **PK própria** em cada tabela, campos obrigatórios (NOT NULL), valores únicos (UNIQUE) e valores padrão (DEFAULT).

4. **As ligações viram colunas.** `matriculas.rm_aluno` guarda o RM do aluno, com o **mesmo tipo** da PK de `alunos`. A FK fica para depois.

O MER tinha só os atributos principais. No banco, completamos: CPF, nascimento, cidade, e-mail, carga horária, nota e faltas.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 2 DE 9

O projeto físico da escola

professores

cod_prof	integer	PK
nome_prof	varchar(100)	NOT NULL
email_prof	varchar(100)	UNIQUE

alunos

rm_aluno	integer	PK
nome_aluno	varchar(100)	NOT NULL
cpf_aluno	char(11)	UNIQUE
data_nasc	date	
cidade_aluno	varchar(60)	

disciplinas

sigla_disc	varchar(10)	PK
nome_disc	varchar(60)	NOT NULL
carga_horaria	integer	NOT NULL
cod_prof	integer	→ professores

matriculas

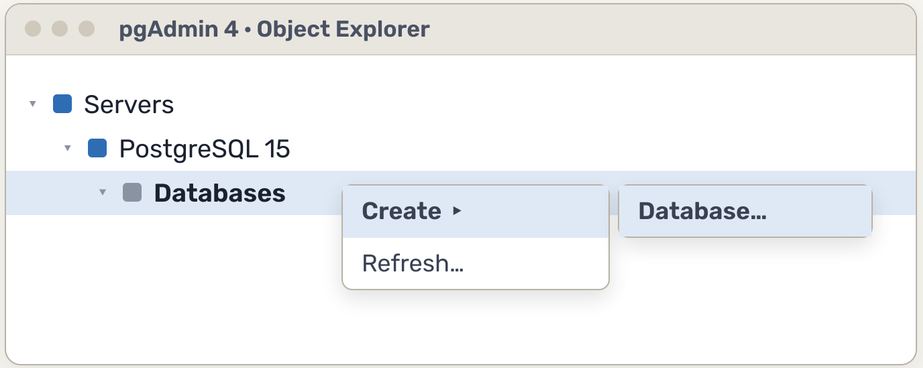
cod_mat	integer	PK
rm_aluno	integer	NOT NULL → alunos
sigla_disc	varchar(10)	NOT NULL → disciplinas
data_mat	date	NOT NULL DEFAULT hoje
nota	numeric(4,2)	
faltas	integer	NOT NULL DEFAULT 0

Toda tabela tem a **sua própria PK**, um código simples. Em `matriculas`, é o `cod_mat`: o número da matrícula.

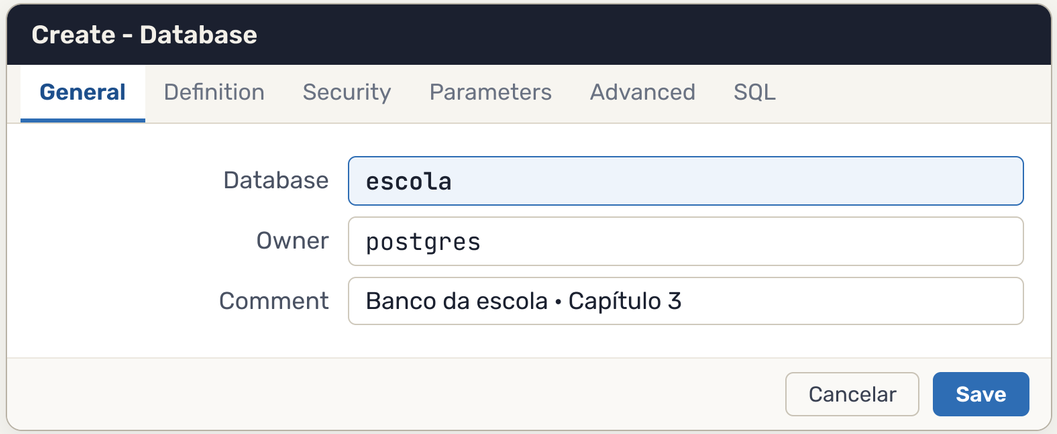
A seta `→ alunos` marca a ligação do MER: a coluna guarda a PK da outra tabela.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 3 DE 9

Criar o banco escola



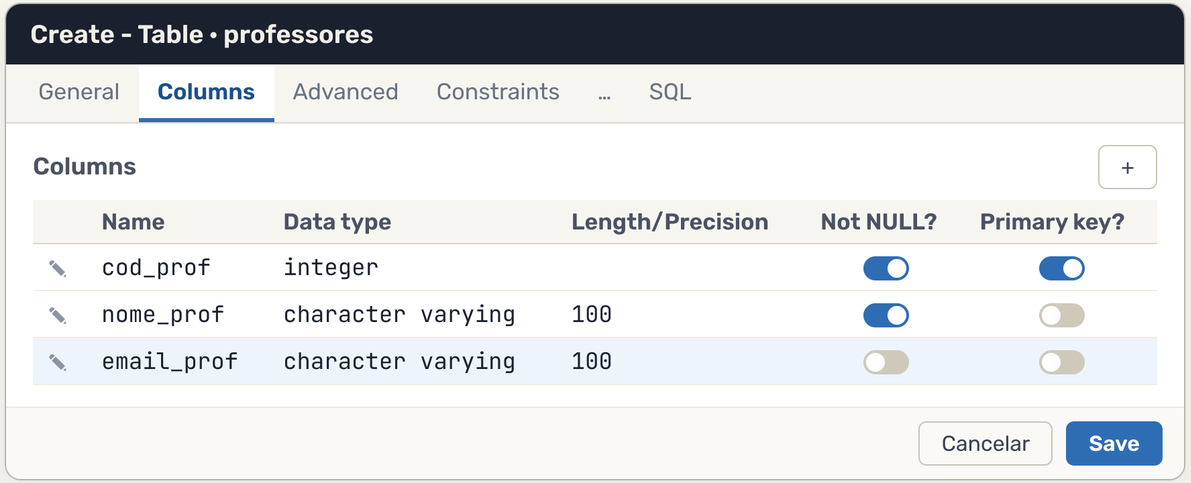
1. Botão direito em **Databases** → **Create** → **Database…**
2. **Database**: `escola`. Owner: o seu usuário (`postgres` ou `avnadmin`).
3. **Save**. O banco aparece em Databases.

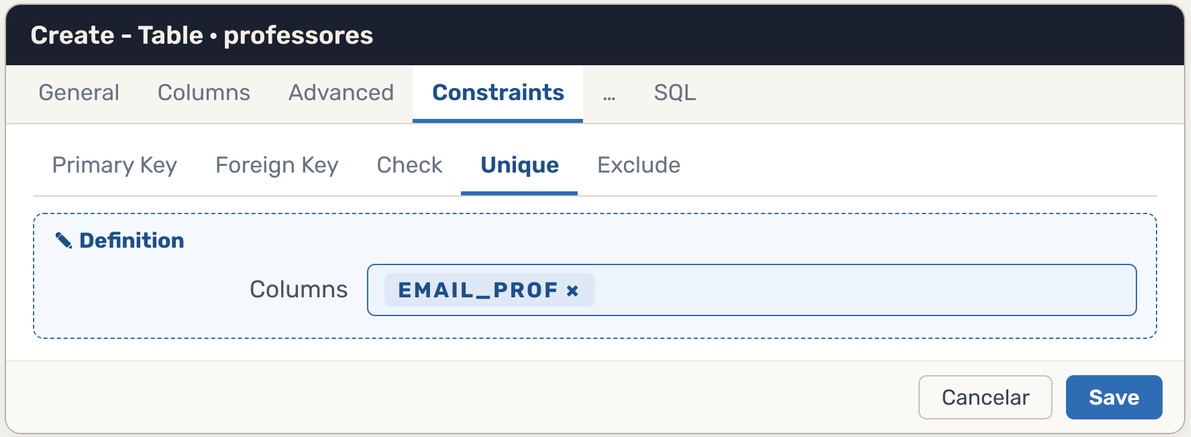


Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 4 DE 9

Tabela professores: colunas, PK e UNIQUE





1. Em **escola** → **Schemas** → **public**: botão direito em **Tables** → **Create** → **Table…** Aba **General**: Name `professores`.
2. Aba **Columns**: **+** para cada coluna — nome, tipo e tamanho (*Length*).
3. Ligue **Primary key?** em `cod_prof` e **Not NULL?** em `nome_prof`.
4. Aba **Constraints** → **Unique** → **+**, lápis, **Definition** → **Columns**: `email_prof`.
5. **Save**.

> No pgAdmin, `varchar` aparece como **character varying**, e `char` como **character**.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 5 DE 9

Tabelas disciplinas e alunos

Create - Table • disciplinas

General | **Columns** | Constraints | … | SQL

Columns +

	Name	Data type	Length/Precision	Not NULL?	Primary key?
✎	sigla_disc	character varying	10	●	●
✎	nome_disc	character varying	60	●	○
✎	carga_horaria	integer		●	○
✎	cod_prof	integer		○	○

Cancelar **Save**

Create - Table • alunos

General | **Columns** | Constraints | … | SQL

Columns +

	Name	Data type	Length/Precision	Not NULL?	Primary key?
✎	rm_aluno	integer		●	●
✎	nome_aluno	character varying	100	●	○
✎	cpf_aluno	character	11	○	○
✎	data_nasc	date		○	○
✎	cidade_aluno	character varying	60	○	○

Cancelar **Save**

Mesmo processo

General → Columns → Save. Nenhum clique novo.

disciplinas.cod_prof

Guarda o código do professor: tipo `integer`, **igual à PK** de professores.

alunos.cpf_aluno

`character` 11 — texto, por causa do zero à esquerda. E **Unique**, como o e-mail do professor.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 6 DE 9

Tabela matriculas: código próprio e DEFAULT

Create - Table • matriculas

General | **Columns** | Advanced | Constraints | … | SQL

Columns +

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?
✎	cod_mat	integer			●	●
✎	rm_aluno	integer			●	○
✎	sigla_disc	character varying	10		●	○
✎	data_mat	date			●	○
✎	nota	numeric	4	2	○	○
✎	faltas	integer			●	○

✎ **data_mat**

General | Definition | **Constraints** | …

Default value: `CURRENT_DATE`

Not NULL? ●

Cancelar | **Save**

Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

1. Crie as 6 colunas. Em `nota`: tipo **numeric**, *Length/Precision* `4`, *Scale* `2`.
2. **PK:** ligue **Primary key?** em `cod_mat`. Em `rm_aluno` e `sigla_disc`, só **Not NULL?**.
3. **DEFAULT:** no lápis ✎ de `data_mat` → aba **Constraints** → *Default value*: `CURRENT_DATE`.
4. No lápis de `faltas`: *Default value* `0`.
5. **Save**.

> `CURRENT_DATE` = a data do dia em que a linha for inserida.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 7 DE 9

Conferir a aba SQL

ALUNOS • ABA SQL, RESUMIDA

```sql
CREATE TABLE IF NOT EXISTS public.alunos
(
    rm_aluno integer NOT NULL,
    nome_aluno character varying(100) NOT NULL,
    cpf_aluno character(11),
    data_nasc date,
    cidade_aluno character varying(60),
    PRIMARY KEY (rm_aluno),
    UNIQUE (cpf_aluno)
);
```

MATRICULAS • ABA SQL, RESUMIDA

```sql
CREATE TABLE IF NOT EXISTS public.matriculas
(
    cod_mat integer NOT NULL,
    rm_aluno integer NOT NULL,
    sigla_disc character varying(10) NOT NULL,
    data_mat date NOT NULL DEFAULT CURRENT_DATE,
    nota numeric(4, 2),
    faltas integer NOT NULL DEFAULT 0,
    PRIMARY KEY (cod_mat)
);
```

Seus cliques viraram SQL

Cada tipo, **NOT NULL**, **PRIMARY KEY**, **UNIQUE** e **DEFAULT** aparece no comando.

É o DDL

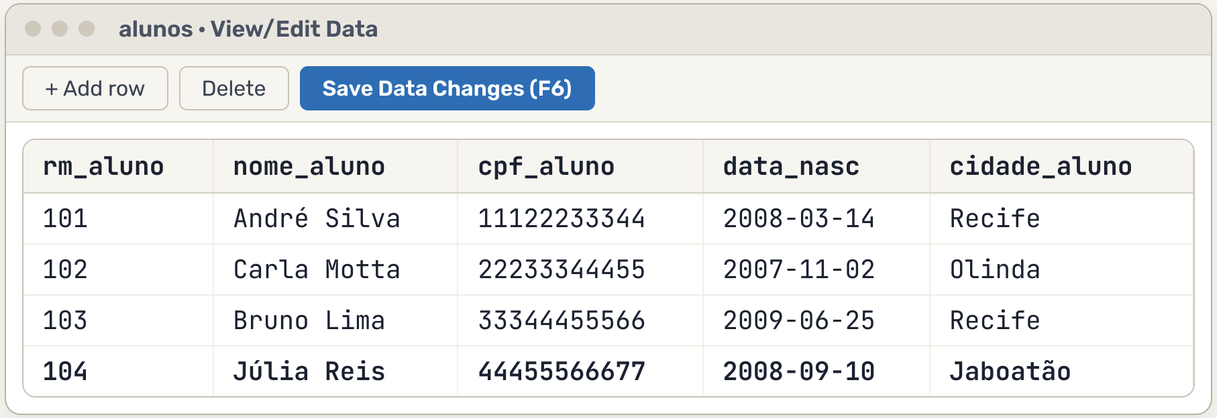
A parte da SQL que **define** a estrutura. Aqui, o pgAdmin escreveu por você.

Revise antes do Save

Faltou um NOT NULL? A PK está na coluna certa? A aba SQL mostra.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 8 DE 9

Lançar os dados pela grade



Esquema simplificado da tela — a aparência muda um pouco entre versões do pgAdmin.

1. Botão direito na tabela → **View/Edit Data** → **All Rows**.
2. **Add row** e digite cada valor na célula.
3. **Save Data Changes** (F6). Sem salvar, nada vai para o banco.

- Datas: `2008-09-10` (ano-mês-dia).
- Decimais com **ponto**: `8.5`.
- Coluna com DEFAULT? Deixe vazia: o banco preenche.
- A grade só deixa editar tabelas **com PK**.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 8 DE 9

Os dados da escola

PROFESSORES

cod_prof	nome_prof	email_prof
7	Rui Lima	rui.lima@escola.edu.br
8	Ana Reis	ana.reis@escola.edu.br

ALUNOS

rm_aluno	nome_aluno	cpf_aluno	data_nasc	cidade_aluno
101	André Silva	11122233344	2008-03-14	Recife
102	Carla Motta	22233344455	2007-11-02	Olinda
103	Bruno Lima	33344455566	2009-06-25	Recife
104	Júlia Reis	44455566677	2008-09-10	Jaboatão
105	Davi Rocha	55566677788	2006-01-30	Recife

DISCIPLINAS

sigla_disc	nome_disc	carga_horaria	cod_prof
BD	Banco de Dados	80	7
LOG	Lógica de Programação	60	8
WEB	Desenvolvimento Web	80	7

MATRICULAS

cod_mat	rm_aluno	sigla_disc	data_mat	nota	faltas
1	101	BD	2026-02-02	8.5	2
2	101	LOG	2026-02-03	7.0	0
3	102	BD	2026-02-02	9.5	1
4	103	WEB	2026-02-05	6.0	10
5	104	BD	2026-02-04	*[null]*	0
6	105	LOG	2026-02-03	5.5	16

[null] = deixe a célula vazia: a nota da Júlia ainda não foi lançada. Comece pelos cadastros e termine nas matrículas.

CONSTRUÇÃO GUIADA • ESCOLA • PASSO 9 DE 9

O banco cobra as regras?

Tentamos lançar dados errados de propósito, na grade. A mensagem de erro diz **qual regra** barrou.

TENTATIVA	REGRA	RESULTADO	MENSAGEM
Um aluno com RM `101` de novo	PRIMARY KEY	✗ recusado	`duplicate key value violates unique constraint "alunos_pkey"`
Um aluno com o CPF do André	UNIQUE	✗ recusado	`duplicate key value violates unique constraint "alunos_cpf_aluno_key"`
Um aluno sem nome	NOT NULL	✗ recusado	`null value in column "nome_aluno" … violates not-null constraint`
Carga horária `oitenta`	tipo integer	✗ recusado	`invalid input syntax for type integer: "oitenta"`
Matrícula sem data e sem faltas	DEFAULT	✓ aceito	`aceita: data de hoje e 0 faltas`
Matrícula do aluno `999`	— nenhuma	✓ aceito	`aceita!`

⚠ A última passou — e não deveria: **não existe aluno 999**. Quem confere essa ligação é a **FOREIGN KEY**, o próximo passo do nosso banco. Por enquanto, quem cuida disso é você.

PRÁTICA 4 • NOVA SITUAÇÃO • 30 MIN • INDIVIDUAL

Agora é com você: a biblioteca

“A biblioteca da escola quer sair do caderno: cadastrar editoras, livros e leitores — e registrar cada empréstimo.”

editoras	
cod_editora	código; identifica a editora
nome_editora	obrigatório
cidade_editora	texto

leitores	
cod_leitor	código; identifica o leitor
nome_leitor	obrigatório
email_leitor	não se repete
turma	ex.: TDS-1

livros	
cod_livro	código; identifica o livro
titulo	obrigatório
isbn	13 dígitos; não se repete
ano_publicacao	ano, ex.: 1899
cod_editora	código da editora do livro

emprestimos	
cod_emprestimo	código; identifica o empréstimo
cod_livro	código do livro
cod_leitor	código do leitor
data_retirada	se não for informada, é hoje
data_devolucao	vazia até a devolução
renovacoes	obrigatório; começa em 0

O QUE FAZER

1. Crie o banco `biblioteca`.
2. Crie as 4 tabelas. **Você decide** o tipo de cada coluna.
3. Pelas regras, configure **PK**, **Not NULL**, **Unique** e **Default**.
4. Confira a aba **SQL** de cada tabela.
5. Lance os dados do próximo slide.

PENSE E RESPONDA Por que `emprestimos` ganhou um código próprio, se já guarda o livro e o leitor?

PRÁTICA 4 • OS DADOS

Os dados para lançar

EDITORAS

cod_editora	nome_editora	cidade_editora
1	Editora Aurora	Recife
2	Editora Horizonte	São Paulo
3	Editora Código	Recife

LEITORES

cod_leitor	nome_leitor	email_leitor	turma
1	Marcos Paulo	marcos@email.com	TDS-1
2	Júlia Reis	julia@email.com	TDS-2
3	Lia Rocha	*[null]*	TDS-1
4	Rui Melo	rui@email.com	ADM-1

LIVROS

cod_livro	titulo	isbn	ano_publicacao	cod_editora
10	Dom Casmurro	9786500000101	1899	1
11	O Cortiço	9786500000102	1890	1
12	Iracema	9786500000103	1865	2
13	Lógica de Programação	9786500000104	2019	3
14	Banco de Dados Essencial	9786500000105	2021	3

EMPRESTIMOS

cod_emprestimo	cod_livro	cod_leitor	data_retirada	data_devolucao	renovacoes
1	10	1	2026-09-01	2026-09-10	0
2	12	1	2026-09-15	*[null]*	1
3	11	2	2026-09-20	*[null]*	0
4	13	4	2026-09-22	2026-09-30	0
5	10	3	2026-10-01	*[null]*	0

[null] = deixe a célula vazia: a Lia não tem e-mail, e três livros ainda não foram devolvidos.

PRÁTICA 5 • 10 MIN • INDIVIDUAL

Tente enganar o SGBD

No banco `biblioteca`, tente lançar cada dado pela grade. **Antes** de salvar, escreva o que você espera; **depois**, o que aconteceu.

a Um livro com ano de publicação `mil novecentos`.

b Um livro com o código `10` de novo.

c Um livro novo com o ISBN de Dom Casmurro.

d Um leitor sem nome.

e Um empréstimo retirado em `2026-02-30`.

f Um leitor com o e-mail da Júlia.

g O empréstimo 6, para o leitor `99` — que não existe.

h O empréstimo 7, informando só o código, o livro e o leitor.

PARA CADA TESTE

```
Esperava: …
Aconteceu: …
Regra que barrou: …
```

A mensagem de erro diz **qual regra** barrou. Leia com calma — ela é sua aliada.

PENSE E RESPONDA Dois testes foram aceitos. Qual deles é um problema? O que faltou no banco para recusá-lo?

SEÇÃO 04

SQL: consulta e manipulação

04

A linguagem dos bancos relacionais: buscar, filtrar e ordenar dados — e inserir, alterar e excluir com segurança.

O que é SQL

SQL — *Structured Query Language*, linguagem de consulta estruturada — é como **conversamos com um SGBD relacional**.

- Criada na IBM nos anos 1970; padrão ANSI e ISO desde os anos 1980.
- A mesma base no PostgreSQL, MySQL, SQL Server e Oracle — cada um com pequenos “sotaques”.
- É a língua comum entre **programadores** e **DBAs**.

PYTHON • LISTA DE DICIONÁRIOS

```python
for aluno in alunos:
    if aluno["cidade_aluno"] == "Recife":
        print(aluno["nome_aluno"])
```

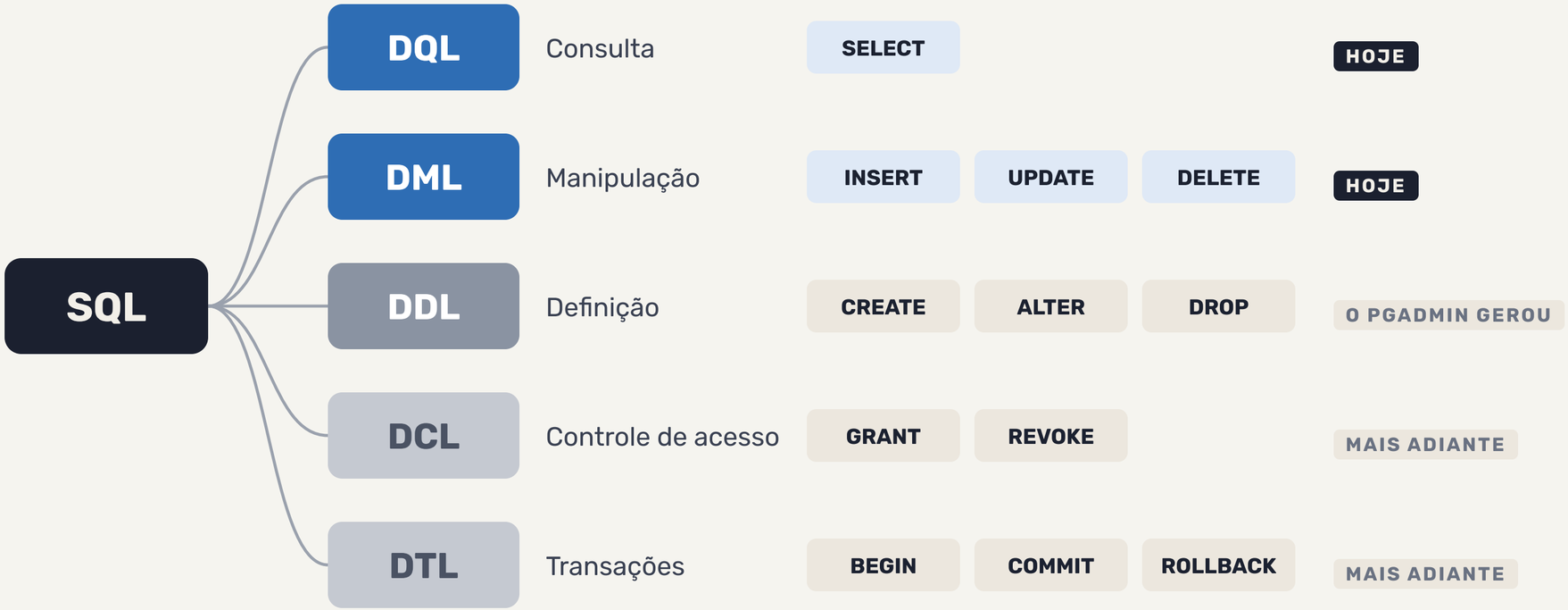
SQL • NO BANCO

```sql
SELECT nome_aluno
FROM alunos
WHERE cidade_aluno = 'Recife';
```

No Python, você diz **como** percorrer. Na SQL, você diz **o que** quer — e o SGBD decide como buscar.

04 • SQL

As famílias de comandos



Hoje: **DQL** (consultar) e **DML** (inserir, alterar, excluir). O **DDL** você já viu na aba SQL do pgAdmin.

04 • SQL

Onde escrever SQL: a Query Tool



Botão direito no banco **escola** → **Query Tool**.

REGRAS DE ESCRITA

Termine com `;`
Cada comando acaba em ponto e vírgula.

Texto e data entre `' '`
`'Recife'`, `'2008-03-14'`. Aspas **simples**.

Número sem aspas
`101`, `8.5` — com **ponto** decimal.

MAIÚSCULAS nas palavras-chave
É convenção: o SGBD aceita `select`, mas fica mais legível.

Comentários
`-- isto não é executado`

Selecionou um trecho?
O F5 executa **só ele**.

04 • SQL • SINTAXE

Anatomia de um comando SQL

```
SELECT nome_aluno, cidade_aluno     ← o que mostrar: as colunas
FROM alunos                         ← de onde: a tabela
WHERE cidade_aluno = 'Recife'       ← quais linhas: a condição
ORDER BY nome_aluno;                ← em que ordem • ; encerra
```

nome_aluno	cidade_aluno
André Silva	Recife
Bruno Lima	Recife
Davi Rocha	Recife

Espaços e quebras de linha **não mudam** o comando. Uma cláusula por linha deixa tudo mais fácil de ler.

AS PEÇAS

`SELECT FROM WHERE` Palavras-chave
Reservadas da linguagem. Em MAIÚSCULAS, por convenção.

`alunos · nome_aluno` Nomes
Tabelas e colunas, escritos como foram criados.

`'Recife'` `101` Valores
Texto e data entre aspas **simples**; número sem aspas.

`= <> > AND OR` Operadores
Comparam e combinam condições.

`;` Terminador
Marca o fim do comando.

04 • SQL • SINTAXE

O molde do SELECT

```
SELECT [DISTINCT] colunas
FROM tabela
[WHERE condição]
[ORDER BY coluna [ASC | DESC]]
[LIMIT n];
```

`MAIÚSCULA` escreva igual — *`itálico`* troque pelo seu — `[ … ]` opcional — `|` escolha um

`SELECT colunas`	`*` = todas; ou os nomes, separados por vírgula. `DISTINCT` tira as linhas repetidas.
`FROM tabela`	De qual tabela vêm os dados. Obrigatório.
`WHERE condição`	Filtra as linhas. Sem ele, vêm **todas**.
`ORDER BY coluna`	Ordena o resultado. `ASC` (padrão) ou `DESC`.
`LIMIT n`	Mostra só as primeiras *n* linhas.

SÓ O OBRIGATÓRIO

```
SELECT * FROM alunos;
```

TODAS AS PARTES

```
SELECT DISTINCT cidade_aluno
FROM alunos
WHERE cidade_aluno <> 'Recife'
ORDER BY cidade_aluno DESC
LIMIT 1;
```

⚠ A **ordem das cláusulas** é fixa: um `WHERE` depois do `ORDER BY` dá erro de sintaxe.

PRÁTICA 6 • 5 MIN • INDIVIDUAL

Leia o comando

Comandos para o banco `biblioteca`. Sem executar: diga a **família** de cada um e o que ele faz, **em português**.

a

```
SELECT titulo FROM livros WHERE
ano_publicacao < 1900;
```

b

```
INSERT INTO leitores (cod_leitor,
nome_leitor) VALUES (5, 'Caio Nunes');
```

c

```
UPDATE emprestimos SET data_devolucao =
'2026-10-14' WHERE cod_emprestimo = 2;
```

d

```
DELETE FROM emprestimos WHERE
data_devolucao IS NOT NULL;
```

e

```
CREATE TABLE autores (cod_autor integer
PRIMARY KEY);
```

f

```
GRANT SELECT ON livros TO estagiario;
```

FORMATO DA RESPOSTA

a) DQL — mostra o título dos livros publicados antes de 1900.

PENSE E RESPONDA Qual desses comandos você pode executar sem medo de estragar nada? Por quê?

04 • SQL • DQL

SELECT: buscar dados

TODAS AS COLUNAS • `*`

```
SELECT * FROM alunos;
```

rm_aluno	nome_aluno	cpf_aluno	data_nasc	cidade_aluno
101	André Silva	11122233344	2008-03-14	Recife
102	Carla Motta	22233344455	2007-11-02	Olinda
103	Bruno Lima	33344455566	2009-06-25	Recife
104	Júlia Reis	44455566677	2008-09-10	Jaboatão
105	Davi Rocha	55566677788	2006-01-30	Recife

SÓ AS COLUNAS QUE INTERESSAM

```
SELECT nome_aluno, cidade_aluno
FROM alunos;
```

nome_aluno	cidade_aluno
André Silva	Recife
Carla Motta	Olinda
Bruno Lima	Recife
Júlia Reis	Jaboatão
Davi Rocha	Recife

Estrutura: `SELECT` quais colunas `FROM` qual tabela. As colunas vêm separadas por vírgula.

O `*` é ótimo para explorar. No sistema, peça **só o que vai usar**.

SELECT: apelidos, cálculos e textos

APELIDO (AS) E CÁLCULO

```sql
SELECT nome_disc AS disciplina,
       carga_horaria,
       carga_horaria * 0.25 AS limite_faltas
FROM disciplinas;
```

disciplina	carga_horaria	limite_faltas
Banco de Dados	80	20.00
Lógica de Programação	60	15.00
Desenvolvimento Web	80	20.00

Limite de faltas: 25% da carga horária. O cálculo **não altera** a tabela — só a resposta.

JUNTAR TEXTOS • CONCATENAÇÃO

```sql
SELECT nome_aluno || ' (RM ' || rm_aluno || ')' AS aluno
FROM alunos;
```

aluno
André Silva (RM 101)
Carla Motta (RM 102)
Bruno Lima (RM 103)
… 5 linhas

```sql
SELECT CONCAT(nome_aluno, ' - ', cidade_aluno)
FROM alunos;
```

No PostgreSQL: o operador `||` ou a função `CONCAT()`. Se um valor for **NULL**, `||` devolve NULL; `CONCAT` ignora.

EXEMPLO EXPLICADO • SELECT

Do pedido ao SELECT: 4 situações

“Em quais cidades temos alunos?”

```
SELECT DISTINCT cidade_aluno
FROM alunos
ORDER BY cidade_aluno;
```

cidade_aluno
Jaboatão
Olinda
Recife

`DISTINCT` tira as repetições: sem ele, Recife viria 3 vezes.

“Quero a lista de chamada: RM e nome, em ordem alfabética.”

```
SELECT rm_aluno || ' - ' ||
       nome_aluno AS chamada
FROM alunos
ORDER BY nome_aluno;
```

chamada
101 - André Silva
103 - Bruno Lima
102 - Carla Motta
… 5 linhas

Concatenar monta um texto novo; `AS` dá nome à coluna.

“Quantas semanas dura cada disciplina, com 4 h por semana?”

```
SELECT sigla_disc,
       carga_horaria / 4 AS semanas
FROM disciplinas;
```

sigla_disc	semanas
BD	20
LOG	15
WEB	20

A conta aparece na resposta, mas **não altera** a tabela.

“Quem é o aluno mais velho?”

```
SELECT nome_aluno, data_nasc
FROM alunos
ORDER BY data_nasc
LIMIT 1;
```

nome_aluno	data_nasc
Davi Rocha	2006-01-30

Data mais antiga primeiro; `LIMIT 1` pega só a primeira linha.

WHERE: filtrar as linhas

`WHERE` filtra **quais linhas** entram na resposta: só as que tornam a condição verdadeira.

OPERADOR	SIGNIFICA	EXEMPLO
`= <>`	igual • diferente	`cidade_aluno = 'Recife'`
`> ⩾ < ⩽`	maior, menor…	`nota ⩾ 7`
`AND OR NOT`	combinar condições	`nota < 7 OR faltas > 5`
`BETWEEN`	dentro de uma faixa	`data_nasc BETWEEN '2008-01-01' AND '2008-12-31'`
`IN`	está na lista	`cidade_aluno IN ('Olinda', 'Jaboatão')`
`LIKE`	padrão de texto • % = qualquer coisa	`nome_aluno LIKE '%Lima'`
`IS NULL`	está vazio	`nota IS NULL`

```
SELECT rm_aluno, sigla_disc, nota
FROM matriculas
WHERE nota ⩾ 7;
```

rm_aluno	sigla_disc	nota
101	BD	8.50
101	LOG	7.00
102	BD	9.50

⚠ A Júlia (nota **NULL**) não aparece: NULL não é ≥ 7 nem < 7. Para achá-la, use `IS NULL`.

WHERE: AND, OR e parênteses

PEDIDO DA COORDENAÇÃO

“Quais alunos de **Lógica** estão em risco: nota abaixo de 7 **ou** mais de 5 faltas?”

SEM PARÊNTESES

```
SELECT cod_mat, sigla_disc, nota, faltas
FROM matriculas
WHERE sigla_disc = 'LOG' AND nota < 7
   OR faltas > 5;
```

cod_mat	sigla_disc	nota	faltas
4	WEB	6.00	10
6	LOG	5.50	16

✗ Veio uma matrícula de **WEB**! O banco leu: (LOG **e** nota < 7) **ou** faltas > 5.

COM PARÊNTESES

```
SELECT cod_mat, sigla_disc, nota, faltas
FROM matriculas
WHERE sigla_disc = 'LOG'
  AND (nota < 7 OR faltas > 5);
```

cod_mat	sigla_disc	nota	faltas
6	LOG	5.50	16

✓ Os parênteses juntam as duas condições do “ou”, e o **LOG** vale para ambas.

AS REGRAS

- `AND`: as duas precisam ser verdade.
- `OR`: basta uma.
- `NOT`: inverte.
- O `AND` é resolvido **antes** do `OR`.
- **Misturou AND e OR? Use parênteses.**

WHERE com textos: =, LIKE e ILIKE

Texto é comparado **letra por letra** — inclusive maiúsculas e acentos. Para buscar por pedaços, use `LIKE`.

MAIÚSCULAS IMPORTAM

```
WHERE cidade_aluno = 'recife'
```

```
0 linhas — "recife" ≠ "Recife"
```

```
WHERE cidade_aluno ILIKE 'recife'
```

```
3 linhas — ILIKE ignora maiúsculas
```

PADRÕES COM LIKE • EM `NOME_ALUNO`

PADRÃO	SIGNIFICA	ENCONTRA
`LIKE 'A%'`	começa com A	André Silva
`LIKE '%Lima'`	termina com Lima	Bruno Lima
`LIKE '%ar%'`	contém “ar”	Carla Motta
`LIKE '_a%'`	a 2ª letra é a	Carla Motta · Davi Rocha

`%` = qualquer sequência, até vazia • `_` = exatamente um caractere.

`cidade_aluno <> 'Recife'`	diferente de	Carla (Olinda) • Júlia (Jaboatão)
`cidade_aluno NOT IN ('Recife', 'Olinda')`	fora da lista	Júlia (Jaboatão)

NULL: o valor que não está lá

NULL não é zero

`faltas = 0` é um valor: o aluno não faltou.

NULL não é texto vazio

`''` é um texto sem letras. NULL é “sem valor”.

NULL é “não sei”

A nota da Júlia ainda **não foi lançada**.

ERRADO • NÃO DÁ ERRO, MAS NÃO ACHA

```sql
SELECT cod_mat, nota
FROM matriculas
WHERE nota = NULL;
```

```
0 linhas
```

CERTO

```sql
SELECT cod_mat, nota
FROM matriculas
WHERE nota IS NULL;
```

cod_mat	nota
5	*[null]*

CONTA COM NULL DÁ NULL

```sql
SELECT cod_mat, nota, nota + 1 AS mais_um
FROM matriculas
WHERE rm_aluno IN (102, 104);
```

cod_mat	nota	mais_um
3	9.50	10.50
5	*[null]*	*[null]*



Comparar com NULL nunca dá “verdadeiro”. Use sempre `IS NULL` e `IS NOT NULL`.

ORDER BY e LIMIT

`ORDER BY` • ORDENAR

```sql
SELECT nome_aluno, data_nasc
FROM alunos
ORDER BY data_nasc;
```

nome_aluno	data_nasc
Davi Rocha	2006-01-30
Carla Motta	2007-11-02
André Silva	2008-03-14
Júlia Reis	2008-09-10
Bruno Lima	2009-06-25

Crescente é o padrão (`ASC`). Para decrescente: `DESC` .

`LIMIT` • AS PRIMEIRAS N LINHAS

```sql
SELECT rm_aluno, sigla_disc, nota
FROM matriculas
WHERE nota IS NOT NULL
ORDER BY nota DESC
LIMIT 3;
```

rm_aluno	sigla_disc	nota
102	BD	9.50
101	BD	8.50
101	LOG	7.00

⚠ Sem o `IS NOT NULL` , a Júlia viria em 1º: no PostgreSQL, o `DESC` coloca os **NULL no topo**.

💡 **Ordem das cláusulas:** SELECT → FROM → WHERE → ORDER BY → LIMIT. Sempre nessa sequência.

04 • SQL • SINTAXE

A ordem em que o banco lê o SELECT

VOCÊ ESCREVE NESTA ORDEM

`SELECT` → `FROM` → `WHERE` → `ORDER BY` → `LIMIT`

O BANCO EXECUTA NESTA

FROM → **WHERE** → **SELECT** → **ORDER BY** → **LIMIT**

- **FROM** — pega a tabela
- **WHERE** — filtra as linhas
- **SELECT** — monta as colunas e os apelidos
- **ORDER BY** — ordena
- **LIMIT** — corta

APELIDO NO WHERE

```sql
SELECT nome_disc, carga_horaria * 0.25 AS limite_faltas
FROM disciplinas
WHERE limite_faltas > 15;
```

```
ERROR: column "limite_faltas" does not exist
```

O WHERE roda **antes** do SELECT: o apelido ainda não existe.

APELIDO NO ORDER BY

```sql
SELECT nome_disc, carga_horaria * 0.25 AS limite_faltas
FROM disciplinas
ORDER BY limite_faltas DESC;
```

nome_disc	limite_faltas
Banco de Dados	20.00
Desenvolvimento Web	20.00
Lógica de Programação	15.00

O ORDER BY roda **depois**: o apelido já existe.

EXEMPLO EXPLICADO • ESTRATÉGIA

Da pergunta ao SELECT

PERGUNTA DA COORDENAÇÃO

“Quais alunos de **Recife** nasceram a partir de **2008**? Quero em ordem alfabética.”

1 De qual tabela?
Os dados de nome, cidade e nascimento estão em alunos.

```
FROM alunos
```

2 Quais linhas?
De Recife e nascidos a partir de 2008: duas condições, com AND.

```
WHERE cidade_aluno = 'Recife'
  AND data_nasc ⩾ '2008-01-01'
```

3 Quais colunas?
O que a pergunta quer ver: nome e data.

```
SELECT nome_aluno, data_nasc
```

4 Em que ordem?
“Em ordem alfabética”: pelo nome.

```
ORDER BY nome_aluno
```

MONTADO NA ORDEM CERTA

```
SELECT nome_aluno, data_nasc
FROM alunos
WHERE cidade_aluno = 'Recife'
  AND data_nasc ⩾ '2008-01-01'
ORDER BY nome_aluno;
```

nome_aluno	data_nasc
André Silva	2008-03-14
Bruno Lima	2009-06-25

PRÁTICA 7 • 10 MIN • INDIVIDUAL

Encontre o erro

No banco `biblioteca`, cada comando tem **um** problema. Rode, leia a mensagem e corrija.

a

```
SELECT titulo ano_publicacao FROM livros;
```

b

```
SELECT * FROM livros WHERE titulo =
"Iracema";
```

c

```
SELECT nome_leitor FROM leitores WHERE
email_leitor = NULL;
```

d

```
SELECT titulo FROM livros ORDER BY titulo
WHERE ano_publicacao < 1900;
```

e

```
INSERT INTO leitores (cod_leitor,
nome_leitor, turma) VALUES (6, 'Caio
Nunes');
```

f

```
UPDATE leitores SET turma = TDS-2 WHERE
cod_leitor = 1;
```

g

```
DELETE emprestimos WHERE cod_emprestimo =
3;
```

h

```
SELECT titulo FROM livros WHERE
ano_publicacao > 2000 AND < 2022;
```

PARA CADA UM

```
Problema: …
Correto:  …
```

A mensagem aponta **onde** o banco se perdeu: comece a procurar ali.

PENSE E RESPONDA Dois desses comandos **não dão erro** — e mesmo assim estão errados. Quais? Por que são os mais perigosos?

PRÁTICA 8 • 15 MIN • INDIVIDUAL

Consultas na biblioteca

No banco `biblioteca`, que você montou na Prática 4: um `SELECT` para cada pergunta. Salve tudo em `consultas_biblioteca.sql`.

1. Todos os dados das editoras.
2. Título e ano de publicação de todos os livros.
3. Os livros publicados antes de 1900.
4. As turmas que têm leitores, **sem repetir**.
5. Os empréstimos que ainda não foram devolvidos.
6. Os livros cujo título **começa com "O"**.
7. Os livros da editora 3, em ordem de título.
8. Os 2 livros mais recentes.

PENSE E RESPONDA Desafio: os empréstimos do leitor 1 que tiveram renovação **ou** ainda não foram devolvidos. Teste com e sem parênteses.

INSERT: o molde e as regras

```
INSERT INTO tabela (coluna1, coluna2, …)
VALUES (valor1, valor2, …)[, (…)];
```

`MAIÚSCULA` escreva igual — *`itálico`* troque pelo seu — `[ … ]` opcional — `|` escolha um

rm_aluno	nome_aluno	cidade_aluno
↓	↓	↓
107	'Caio Nunes'	'Paulista'

- **Mesma quantidade, mesma ordem:** o 1º valor vai para a 1ª coluna, e assim por diante.
- **Texto e data** entre aspas simples; data no formato `'AAAA-MM-DD'`.
- **Número e NULL** sem aspas; decimal com ponto: `8.5`.
- **Coluna fora da lista** recebe o DEFAULT — ou fica NULL.
- **Sem a lista de colunas**, o VALUES precisa de todas, na ordem da tabela. Funciona, mas quebra se a tabela mudar: **prefira listar**.

ERROS COMUNS

```
VALUES (107, 'Caio Nunes', 'Paulista')  -- só 2 colunas
```

```
ERROR: INSERT has more expressions than target columns
```

```
VALUES (107, Caio Nunes, 'Paulista')
```

```
ERROR: syntax error at or near "Nunes"
```

```
VALUES (107, 'Caio Nunes', '21/08/2007')
```

```
ERROR: date/time field value out of range: "21/08/2007"
```

Dica: termine com `RETURNING *` para ver a linha que entrou.

INSERT: inserir linhas

UMA LINHA, TODAS AS COLUNAS

```
INSERT INTO alunos
       (rm_aluno, nome_aluno, cpf_aluno,
        data_nasc, cidade_aluno)
VALUES (106, 'Lia Rocha', '66677788899',
        '2009-04-18', 'Olinda');
```

```
INSERT 0 1    ← 1 linha inserida
```

- Liste as **colunas** e depois os **valores**, na mesma ordem.
- Várias linhas num comando só: separe os grupos por vírgula.
- Coluna omitida recebe o **DEFAULT** — data de hoje, 0 faltas — ou fica **NULL**.

VÁRIAS LINHAS, COLUNAS OMITIDAS

```
INSERT INTO matriculas
       (cod_mat, rm_aluno, sigla_disc)
VALUES (7, 106, 'BD'),
       (8, 106, 'WEB');
```

cod_mat	rm_aluno	sigla_disc	data_mat	nota	faltas
7	106	BD	2026-10-15	*[null]*	0
8	106	WEB	2026-10-15	*[null]*	0

data_mat = data do dia · faltas = 0 (DEFAULT)

⚠ As regras valem aqui também: RM repetido, CPF repetido, aluno sem nome ou texto num campo numérico são **recusados**, como na grade.

UPDATE: o molde e quatro situações

```
UPDATE tabela
SET coluna = novo_valor[, coluna2 = valor2]
WHERE condição;
```

`MAIÚSCULA` escreva igual `itálico` troque pelo seu `[ … ]` opcional `|` escolha um

- `SET` diz **o que muda**; várias colunas vão separadas por vírgula.
- `WHERE` diz **quais linhas** mudam. Para uma linha só, use a **PK**.
- No `SET`, o `=` **atribui**; no `WHERE`, ele **compara**.
- A resposta `UPDATE n` conta as linhas alteradas. Esperava 1 e veio 7? Algo deu errado.

SITUAÇÃO	SET	WHERE	RETORNO
Corrigir um dado de uma linha “O Rui trocou de e-mail.”	`email_prof = 'rui.lima@novaescola.edu.br'`	`cod_prof = 7`	`UPDATE 1`
Mudar um grupo “As disciplinas de 80 h passam a 100 h.”	`carga_horaria = 100`	`carga_horaria = 80`	`UPDATE 2`
Calcular a partir do atual “Meio ponto extra em BD para quem já tem nota.”	`nota = nota + 0.5`	`sigla_disc = 'BD' AND nota IS NOT NULL`	`UPDATE 2`
Apagar só um valor “O CPF do Davi está errado: deixe em branco.”	`cpf_aluno = NULL`	`rm_aluno = 105`	`UPDATE 1`

UPDATE: alterar dados

`UPDATE` tabela `SET` coluna = novo valor `WHERE` quais linhas.

LANÇAR UMA NOTA

```sql
UPDATE matriculas
SET nota = 6.5
WHERE cod_mat = 5;    -- Júlia em BD
```

```
UPDATE 1
```

VÁRIAS COLUNAS DE UMA VEZ

```sql
UPDATE alunos
SET cidade_aluno = 'Paulista', data_nasc = '2008-09-11'
WHERE rm_aluno = 104;
```

```
UPDATE 1
```

CALCULAR A PARTIR DO VALOR ATUAL

```sql
UPDATE matriculas
SET faltas = faltas + 2
WHERE sigla_disc = 'WEB';
```

```
UPDATE 2    ← todas as matrículas de WEB
```

cod_mat	rm_aluno	sigla_disc	faltas
4	103	WEB	12
8	106	WEB	2

O `WHERE` decide **quantas** linhas mudam: uma, várias — ou todas.

04 • SQL • DML • SINTAXE

DELETE: o molde e três situações

```
DELETE FROM tabela
WHERE condição;
```

`MAIÚSCULA` escreva igual *`itálico`* troque pelo seu

`[ … ]` opcional `|` escolha um

Não se lista coluna: o `DELETE` sempre apaga a **linha inteira**.

NÃO CONFUNDA

QUERO…	USE
Apagar uma linha	`DELETE … WHERE`
Apagar só o valor de uma coluna	`UPDATE … SET coluna = NULL`
Esvaziar a tabela inteira	`DELETE FROM tabela;` ⚠
Apagar a tabela e a estrutura	`DROP TABLE` — é DDL, não DML

Uma linha, pela PK

“A matrícula 6 foi lançada errada.”

```
DELETE FROM matriculas
WHERE cod_mat = 6;
```

`DELETE 1`

Um grupo, por condição

“WEB foi cancelada: tire as matrículas dela.”

```
DELETE FROM matriculas
WHERE sigla_disc = 'WEB';
```

`DELETE 1`

Filhos antes do pai

“O André saiu da escola.”

```
DELETE FROM matriculas WHERE rm_aluno =
101;
DELETE FROM alunos WHERE rm_aluno = 101;
```

`DELETE 2` `DELETE 1`

04 • SQL • DML

DELETE: excluir linhas

`DELETE FROM` tabela `WHERE` quais linhas. Apaga a **linha inteira** — não existe “apagar só uma coluna”.

DAVI TRANCOU LÓGICA

```
DELETE FROM matriculas
WHERE cod_mat = 6;    -- Davi em LOG
```

```
DELETE 1
```

Para esvaziar só um campo, é `UPDATE … SET nota = NULL`.

EXCLUIR O ANDRÉ?

```
DELETE FROM alunos
WHERE rm_aluno = 101;
```

```
DELETE 1
```

```
SELECT cod_mat, rm_aluno, sigla_disc, nota
FROM matriculas
WHERE rm_aluno = 101;
```

cod_mat	rm_aluno	sigla_disc	nota
1	101	BD	8.50
2	101	LOG	7.00

⚠ O banco deixou — e as matrículas do André ficaram **órfãs**, de um aluno que não existe mais. Sem FK, a ordem é com você: **primeiro as matrículas, depois o aluno**.

04 • SQL • ATENÇÃO

O perigo do UPDATE e do DELETE sem WHERE

```
UPDATE matriculas SET nota = 0;
```

```
UPDATE 7    ← todas as notas viraram zero
```

```
DELETE FROM alunos;
```

```
sem WHERE = todas as linhas
```

⚠ **Não existe Ctrl+Z no banco.** Um `UPDATE` ou `DELETE` sem `WHERE` vale para **todas as linhas** da tabela — e o PostgreSQL não pergunta "tem certeza?".

ESTRATÉGIA: TESTE O WHERE ANTES

1 Escreva o SELECT

```
SELECT * FROM matriculas
WHERE sigla_disc = 'WEB';
```

2 Confira as linhas

```
-- são exatamente
-- estas linhas que
-- devem mudar?
```

3 Troque o começo

```
UPDATE matriculas
SET faltas = faltas + 2
WHERE sigla_disc = 'WEB';
```

EXEMPLO EXPLICADO • MANIPULAÇÃO

Um dia na secretaria

1 “Chegou uma aluna nova: Lia Rocha, RM 106, de Olinda, nascida em 18/04/2009. Ela vai cursar BD.”

```sql
INSERT INTO alunos (rm_aluno, nome_aluno,
data_nasc, cidade_aluno)
VALUES (106, 'Lia Rocha', '2009-04-18',
'Olinda');

INSERT INTO matriculas (cod_mat, rm_aluno,
sigla_disc)
VALUES (7, 106, 'BD');
```

Primeiro o aluno, depois a matrícula: não se matricula quem não está cadastrado.

2 “A Júlia se mudou para Paulista.”

```sql
SELECT * FROM alunos WHERE rm_aluno = 104;
-- conferir

UPDATE alunos SET cidade_aluno = 'Paulista'
WHERE rm_aluno = 104;
```

Identifique a linha pela **PK**, não pelo nome: pode haver dois “André Silva”.

3 “O Davi trancou Lógica.”

```sql
SELECT cod_mat FROM matriculas
WHERE rm_aluno = 105 AND sigla_disc = 'LOG';
-- resposta: 6

DELETE FROM matriculas
WHERE cod_mat = 6;
```

Descubra a **PK** com um SELECT e use **ela** no DELETE: sai exatamente aquela linha.

PRÁTICA 9 • 15 MIN • INDIVIDUAL

Manipulação na biblioteca

Ainda no banco `biblioteca`. Antes de cada `UPDATE` e `DELETE`, rode o `SELECT` com o mesmo `WHERE`.

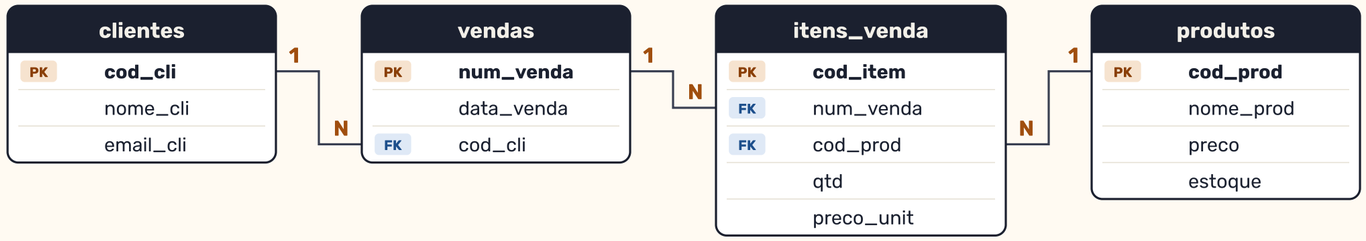
1. Cadastre a editora 4: **Editora Nordeste**, de Fortaleza.
2. Cadastre o livro 15: **Algoritmos em Python**, 2024, ISBN `9786500000106`, editora 4.
3. Registre o empréstimo 6: livro 15 para a Júlia (2), informando **só** o código, o livro e o leitor.
4. A Lia devolveu Dom Casmurro hoje (empréstimo 5): registre a devolução com `CURRENT_DATE`.
5. O Marcos renovou o empréstimo 2: **some 1** às renovações.
6. O Rui (4) mudou de turma: agora é **ADM-2**.
7. O empréstimo 3 foi lançado por engano: remova.
8. O Rui Melo (4) saiu da escola. Exclua o leitor — na **ordem certa**.

PENSE E RESPONDA Depois da tarefa 3, consulte o empréstimo 6. De onde vieram a data de retirada e as renovações?

PRÁTICA 10 • DESAFIO • 30 MIN • INDIVIDUAL

Do MER ao banco: a papelaria

MER • A PLANILHA DO CAPÍTULO 2, NORMALIZADA



Num_Venda	Data	Cod_Cli	Nome_Cli	Produtos (qtd × preço)
1	10/03	5	Lia Rocha	Caderno (2 × 15,00); Caneta (3 × 2,50)
2	10/03	8	Rui Melo	Caneta (1 × 2,50)
3	11/03	5	Lia Rocha	Mochila (1 × 89,90); Caderno (1 × 15,00)

1. No pgAdmin: banco `papelaria` com as 4 tabelas — tipos, uma PK própria em cada tabela (em `itens_venda`: `cod_item`), NOT NULL, UNIQUE (e-mail do cliente) e DEFAULT (estoque 0, data da venda = hoje).

2. Com `INSERT`: os dados da planilha (datas de 2026). Crie os códigos dos produtos.

3. `SELECT`: produtos acima de R$ 10 • vendas da Lia por data • itens da venda 1 com `qtd * preco_unit AS subtotal`.

4. `UPDATE`: a caneta passa a custar 3,00. `DELETE`: tire a mochila da venda 3.

PENSE E RESPONDA Por que `itens_venda` guarda o `preco_unit` se o preço já está em `produtos`?

04 • SQL • CONSULTA RÁPIDA

Cola de sintaxe: os quatro comandos

SELECT • CONSULTAR

```
SELECT colunas
FROM tabela
[WHERE condição]
[ORDER BY col]
[LIMIT n];
```

EXEMPLO

```
SELECT nome_aluno
FROM alunos
WHERE cidade_aluno
      = 'Recife';
```

INSERT • INSERIR

```
INSERT INTO tabela
  (col1, col2)
VALUES (v1, v2);
```

EXEMPLO

```
INSERT INTO professores
  (cod_prof, nome_prof)
VALUES
  (9, 'Marcos Paulo');
```

UPDATE • ALTERAR

```
UPDATE tabela
SET coluna = valor
WHERE condição;
```

EXEMPLO

```
UPDATE matriculas
SET nota = 6.5
WHERE cod_mat = 5;
```

DELETE • EXCLUIR

```
DELETE FROM tabela
WHERE condição;
```

EXEMPLO

```
DELETE FROM matriculas
WHERE cod_mat = 6;
```

REGRAS DE OURO

- **texto e data:** `'aspas simples'`
- **número e NULL: sem aspas**
- **decimal com ponto:** `8.5`
- **data:** `'AAAA-MM-DD'`
- **vazio:** `IS NULL` **, nunca** `= NULL`
- **AND com OR: parênteses**
- **UPDATE e DELETE: sempre WHERE**
- **linha certa: use a PK**

04 • SQL

Confira antes de chamar o professor

SERVIDOR E PGADMIN

- [ ] Servidor rodando: local ou Aiven.
- [ ] Dados de conexão guardados em local seguro.
- [ ] pgAdmin conectado ao servidor.

CONSTRUÇÃO

- [ ] Banco `escola` montado junto com o professor.
- [ ] Banco `biblioteca`: tipos, PK, NOT NULL, UNIQUE e DEFAULT.
- [ ] Dados lançados e salvos nos dois bancos.

SQL

- [ ] Consultas salvas em `consultas_biblioteca.sql`.
- [ ] Todo UPDATE e DELETE com WHERE — testado antes com SELECT.
- [ ] Desafio da papelaria.

BANCO DE DADOS · CONSULTA RÁPIDA

Resumo

Banco de dados
dados relacionados
sobre um assunto
com um propósito

SGBD
software entre
dados e aplicações
integridade · segurança

PostgreSQL
SGBD relacional
gratuito e aberto
cliente/servidor :5432

Local ou Aiven
host · porta · banco
usuário · senha
Aiven: sslmode=require

pgAdmin
cliente gráfico
Create → Table…
View/Edit Data · F6

Tipos
integer · numeric
varchar · char · date
conta? número

Restrições
PRIMARY KEY · NOT NULL
UNIQUE · DEFAULT
FK e CHECK: depois

Famílias SQL
DQL: SELECT
DML: INSERT
UPDATE · DELETE

SELECT
SELECT [DISTINCT] colunas
FROM tabela
AS · || · cálculos

Filtrar e ordenar
WHERE … AND (… OR …)
LIKE · ILIKE · IS NULL
ORDER BY … DESC · LIMIT

INSERT • UPDATE
INSERT INTO … VALUES
UPDATE … SET
… WHERE

DELETE
DELETE FROM …
WHERE sempre
filhos antes do pai

Armadilhas: Query Tool no banco errado • texto sem aspas simples (ou com aspas duplas) • `= NULL` em vez de IS NULL • AND com OR sem parênteses • vírgula decimal • esquecer o F6 • UPDATE/DELETE sem WHERE

ATIVIDADE DE CONSOLIDAÇÃO • INDIVIDUAL

Do briefing ao banco

Barbearia Navalha de Ouro: modelagem, implementação e SQL — tudo junto, numa situação nova.

1 Modelar

DER e MER no draw.io, a partir do briefing

`40 min`

→

2 Implementar

o banco no pgAdmin, com os dados do caderno

`40 min`

→

3 Consultar

DQL: as perguntas do dono

`30 min`

→

4 Manipular

DML: o dia a dia da barbearia

`30 min`

Usa os Capítulos 1, 2 e 3: entidades, cardinalidade, DER, MER, formas normais, pgAdmin, SELECT, INSERT, UPDATE e DELETE.

CONSOLIDAÇÃO • O BRIEFING

Barbearia Navalha de Ouro

MENSAGEM DO DONO DA BARBEARIA

“Aqui na Navalha de Ouro a gente marca horário num caderno e pelo WhatsApp — e vive dando confusão. Quero um sistema simples.

Dos **clientes**, preciso do nome, do telefone — cada cliente tem um, e não pode repetir — e da data de nascimento, para mandar desconto no aniversário. Nem todo mundo informa a data.

Somos três **barbeiros**. De cada um, guardo o nome, o apelido pelo qual os clientes chamam e a data em que começou aqui.

Os **serviços** têm nome, preço e duração em minutos: corte, barba, sobrancelha, pigmentação, hidratação…

Cada **agendamento** é de um cliente, com um barbeiro, num dia e horário. Um mesmo agendamento pode ter vários serviços — corte e barba, por exemplo — e preciso guardar quanto cobrei em cada um, porque às vezes dou desconto.

Todo agendamento começa como “agendado”; depois vira “concluído” ou “cancelado”.”

REGRAS DO PROJETO

- Nomes em minúsculas, com `_`, sem acento.
- Uma **PK própria** em cada tabela: `cod_…`
- DER **sem atributos**; os atributos ficam no MER.
- No MER, marque PK e FK.
- No pgAdmin: tipos, PK, Not NULL, Unique e Default. A FK fica para depois.

CONSOLIDAÇÃO • ETAPA 1 DE 4 • MODELAGEM • 40 MIN

Modele a barbearia: DER e MER

1. Liste as **entidades** e os **atributos** de cada uma: os substantivos do briefing.
2. Encontre os **relacionamentos** — os verbos — e faça as duas perguntas em cada um: 1:1, 1:N ou N:N?
3. **DER no draw.io:** retângulos, losangos com verbo e 1/N nas pontas. Sem atributos.
4. **MER:** uma tabela por entidade, com todos os atributos, PK própria e FK em cada ligação.
5. O **N:N** vira uma tabela nova, com PK própria e os atributos do relacionamento.
6. **Confira as formas normais:** algum campo guarda dois valores? Algum dado se repete?

PISTAS NO BRIEFING

“pode ter vários serviços”
→ que tipo de relacionamento?

“quanto cobrei em cada um”
→ atributo de quem?

“não pode repetir”
→ que regra no banco?

“nem todo mundo informa”
→ campo obrigatório ou não?

“começa como agendado”
→ que regra no banco?

PENSE E RESPONDA Por que o preço cobrado não pode ficar só na tabela de serviços?

CONSOLIDAÇÃO • ETAPA 2 DE 4 • IMPLEMENTAÇÃO • 40 MIN

Implemente o banco no pgAdmin

1. No seu servidor, crie o banco `barbearia`.
2. Crie as tabelas do **seu MER**, no schema `public`.
3. Em cada tabela: **tipos**, **Primary key?**, **Not NULL?**, **Unique** e **Default** — conforme o briefing.
4. Antes de salvar, confira a aba **SQL**.
5. Lance os dados dos próximos dois slides — pela grade ou com `INSERT`.
6. **Teste o DEFAULT:** lance um agendamento sem informar a situação. Veio “agendado”?

DECISÕES DE TIPO

O DADO	SUGESTÃO
Dinheiro	`numeric(6,2)` → `35.00`
Data	`date` → `'2001-10-30'`
Dia e horário juntos	`timestamp` → `'2026-10-28 15:00'`
Telefone	`texto: varchar(20)`
Duração em minutos	`integer`
Situação	`texto, sempre: agendado · concluido · cancelado`

Situação sem acento e em minúsculas: o banco compara texto letra por letra.

CONSOLIDAÇÃO • ETAPA 2 • OS DADOS (1/2)

Os cadastros

As anotações do caderno — do jeito que o dono escreve. **Você converte** para o formato do banco.

CLIENTES

nº	cliente	telefone	nascimento
1	Lucas Andrade	(81) 98888-1001	30/10/2001
2	Pedro Henrique	(81) 98888-1002	12/05/1998
3	Rafael Costa	(81) 98888-1003	02/10/2004
4	Thiago Melo	(81) 98888-1004	20/01/1995
5	Gabriel Rocha	(81) 98888-1005	— não informou
6	Mateus Lima	(81) 98888-1006	07/07/2003

BARBEIROS

nº	barbeiro	apelido	desde
1	Roberto Alves	Beto	01/03/2019
2	Eduardo Lima	Duda	15/08/2021
3	Carlos Souza	Kaká	10/02/2024

TABELA DE SERVIÇOS

nº	serviço	preço	duração
1	Corte	R$ 35,00	30 min
2	Barba	R$ 25,00	20 min
3	Sobrancelha	R$ 10,00	10 min
4	Pigmentação	R$ 40,00	30 min
5	Hidratação	R$ 30,00	20 min

AO LANÇAR

- O **nº** de cada lista é o código: use como PK.
- Datas: `'AAAA-MM-DD'`. Dinheiro: `35.00`.
- “Não informou” = deixe vazio (NULL).

CONSOLIDAÇÃO • ETAPA 2 • OS DADOS (2/2)

A agenda do caderno

AGENDA DA SEMANA • OUTUBRO DE 2026

nº	dia	hora	cliente	barbeiro	serviços (preço cobrado)	situação
1	26/10	09:00	Lucas Andrade	Beto	Corte R$ 35 + Barba R$ 25	concluído
2	26/10	10:00	Pedro Henrique	Duda	Corte R$ 35	concluído
3	26/10	14:30	Rafael Costa	Beto	Corte R$ 35 + Sobrancelha R$ 10	concluído
4	27/10	09:30	Thiago Melo	Kaká	Barba R$ 20 (dei desconto)	concluído
5	27/10	16:00	Gabriel Rocha	Duda	Corte R$ 35 + Pigmentação R$ 40	cancelado
6	28/10	11:00	Lucas Andrade	Kaká	Hidratação R$ 30	agendado
7	28/10	15:00	Mateus Lima	Beto	Corte R$ 35 + Barba R$ 25 + Sobrancelha R$ 10	agendado
8	29/10	10:30	Pedro Henrique	Beto	Barba R$ 25	agendado

Os serviços de cada agendamento também ganham código: numere em sequência, de cima para baixo. O corte do agendamento 1 é o item 1; a barba, o item 2; e assim por diante.

Atenção ao agendamento 4: a barba custa R$ 25 na tabela, mas foi cobrada por R$ 20. Lance o valor **cobrado**.

CONSOLIDAÇÃO • ETAPA 3 DE 4 • CONSULTAS (DQL) • 30 MIN

As perguntas do dono

O dono quer respostas. Um `SELECT` para cada pergunta, salvo em `barbearia.sql` com um comentário: `-- Etapa 3 · 1`.

1. A tabela de preços, do serviço mais caro ao mais barato.
2. Os serviços que custam até R$ 30.
3. Os 3 serviços mais rápidos.
4. Os clientes que não informaram a data de nascimento.
5. Os clientes com sobrenome Lima.
6. Os agendamentos do dia 28/10, em ordem de horário.
   dica: um intervalo com AND
7. Os agendamentos que ainda estão como “agendado”.
8. Todos os agendamentos do Beto.
   dica: descubra o código dele primeiro
9. Os serviços do agendamento 7 e quanto ficaria cada um com 10% de gorjeta.
10. A lista de contatos — “nome - telefone” — em ordem alfabética.

PENSE E RESPONDA

Desafio: os agendamentos **concluídos** do Duda **ou** do Kaká. Rode com e sem parênteses — o que muda?

CONSOLIDAÇÃO • ETAPA 4 DE 4 • MANIPULAÇÃO (DML) • 30 MIN

O dia a dia da barbearia

Continue no `barbearia.sql`. Antes de cada `UPDATE` e `DELETE`, rode o `SELECT` com o mesmo `WHERE`.

1. **Novo cliente:** Vinícius Prado, (81) 98888-1007, nascido em 15/03/2006.
2. O Vinícius marcou com o **Duda** em 30/10 às 17:00: corte e sobrancelha, preço de tabela. É o agendamento 9 — **não informe a situação**.
3. O agendamento 6 — a hidratação do Lucas — foi **concluído**.
4. O Pedro **cancelou** o horário de 29/10.
5. O corte ficou **10% mais caro**.
6. O Thiago trocou de telefone: (81) 97777-2004.
7. Tente cadastrar um cliente com o **telefone do Lucas**. O que aconteceu? Qual regra barrou?
8. **Faxina:** exclua os agendamentos cancelados — e os serviços deles — na **ordem certa**.

PENSE E RESPONDA Depois da tarefa 5, os cortes que já foram feitos mudaram de preço? Por quê?

CONSOLIDAÇÃO • ENTREGA

Entrega e autoconferência

O QUE ENTREGAR

- [ ] `barbearia_der.png` e `barbearia_mer.png`, exportados do draw.io.
- [ ] O banco `barbearia` no seu servidor — local ou Aiven.
- [ ] `barbearia.sql` com as consultas e os comandos, cada um com seu comentário.

O PROFESSOR VAI CONFERIR

- [ ] DER e MER coerentes com o briefing: entidades, cardinalidades, PK e FK.
- [ ] Tipos, PK, Not NULL, Unique e Default no banco.
- [ ] Dados lançados corretamente — datas e valores convertidos.
- [ ] Consultas e comandos funcionando.

AUTOCONFERÊNCIA

- [ ] Rode `SELECT * FROM agendamentos ORDER BY data_hora;`
- [ ] Deu certo se sobraram **7 agendamentos**: 5 concluídos e 2 agendados.
- [ ] E **12 serviços** lançados na tabela dos itens.

Travou numa etapa? Volte ao capítulo: Etapa 1 → Capítulos 1 e 2 • Etapa 2 → Seção 03 • Etapas 3 e 4 → Seção 04 e a cola de sintaxe.

BANCO DE DADOS · CAPÍTULO CONCLUÍDO

Seu banco está no ar!

Entregue a consolidação da Navalha de Ouro. Dúvidas? Discord aberto.

DISCORD

discord.gg/wt5CVZZWJs

DRIVE

drive.google.com/drive/folders/1hOl0DaPeAor7gnhKBUIlZ5n8lLRDNvUY

GITHUB

github.com/TarikPonciano/Programador-de-Sistema-SENAC

PARA ESTUDAR

postgresql.org/docs — documentação oficial do PostgreSQL

pgadmin.org/docs — documentação do pgAdmin 4

aiven.io/docs — PostgreSQL na Aiven

w3schools.com/sql — exercícios rápidos de SQL